



# **Graduation Project**

## **Final Report**

**Project Title: ONLINE MEDICAL SYSTEM**

### **Team 54**

#### **Project Team:**

**Ahmed Mohamed Fathy Shouman**  
**Ahmed Hamdy Mohamed Habib**  
**Ahmed Okasha Nasr El din**  
**Ahmed Mohamed Emam**  
**Ashraf Karam Salem**  
**Abdelrahman mohamed sabry**  
**Ahmed Zedan Abdelrahman**  
**Mohamed Ismael Ibrahim**  
**Abdallah ebrahim Abdelnaby**

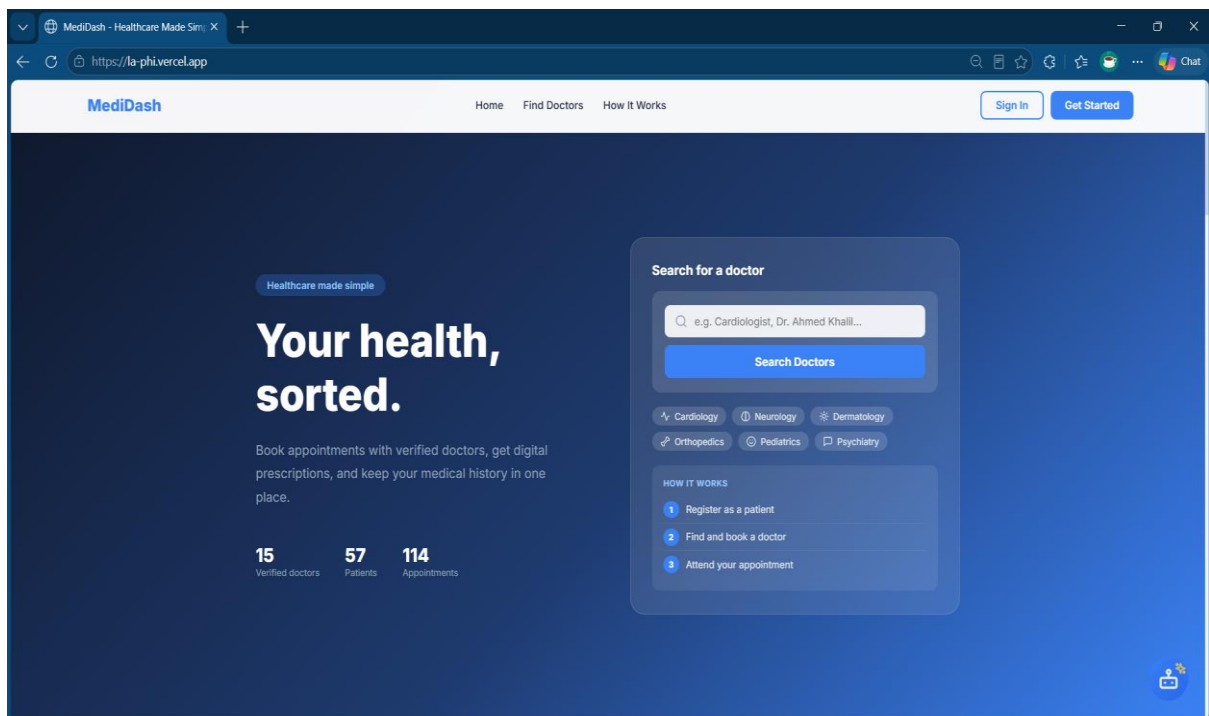
**Academic supervisor: Dr. Salah Eliwa**  
**Assisting supervisor: Eng. Amar Eid**

# ONLINE MEDICAL SYSTEM

## MediDash

### *Project Documentation*

*A comprehensive digital healthcare management platform*



## Acknowledgments

First and foremost, we extend our sincere gratitude to our project supervisor, [DR . Salah Eliwa ], for their invaluable guidance, expert advice, and continuous encouragement throughout every phase of this project, and to [ Eng. Amar Eid ] ; our assistant supervisor who supported us to achieve our academic goals.

Their deep expertise and dedication have been instrumental in shaping the outcome of this work.

We would also like to thank the faculty members of the Computer Science Department for equipping us with the knowledge and skills necessary to undertake a project of this scope. Their support throughout our academic journey has been greatly appreciated.

Special thanks are owed to the doctors and patients who volunteered their time during User Acceptance Testing. Their honest feedback and practical insights directly improved the usability and quality of the final system.

Finally, we wish to express our deep appreciation to our families and friends, whose unwavering support, patience, and encouragement sustained us through the demanding process of bringing this project to life.

## Abstract

The Online Medical System is a comprehensive digital healthcare management platform designed to address persistent challenges in traditional healthcare delivery. In an era of rapid digital transformation, patients face issues such as data loss risks, redundancy, difficult retrieval of medical records, inefficient manual appointment booking, information gaps about available services, and memory challenges for elderly patients regarding medications and schedules.

This project introduces an integrated, user-friendly platform that unifies key services: smart appointment scheduling with doctor selection based on specialty, rating, and cost; virtual consultations (video/audio/chat); centralized electronic health records (EHR) with role-based access; electronic prescriptions with digital signatures; secure payments; and real-time notifications/reminders.

Developed using the Agile methodology for iterative development, continuous feedback, and adaptability to evolving needs, the system employs a modern tech stack: React for web front-end, Flutter for cross-platform mobile app, ASP.NET Core for back-end, and SQL Server for reliable data management – ensuring high performance, security (HIPAA-compliant standards), and scalability.

# Contents

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Acknowledgments.....</b>                     | <b>3</b>  |
| <b>Abstract .....</b>                           | <b>4</b>  |
| <b>Table of Figures .....</b>                   | <b>8</b>  |
| <b>Chapter One: Introduction .....</b>          | <b>10</b> |
| 1.1 Introduction .....                          | 11        |
| <b>1.2 Project Objectives .....</b>             | <b>11</b> |
| 1.3 Project Development Methodology.....        | 12        |
| 1.3.1 Software Development Life Cycle .....     | 12        |
| Main Phases of SDLC.....                        | 12        |
| 1.3.2 Popular SDLC Models .....                 | 13        |
| 1.3.3 Iterative Model.....                      | 13        |
| 1.3.4 Agile Methodology .....                   | 14        |
| 1.3.5 When to Use Agile .....                   | 14        |
| 1.3.6 Why We Chose Agile Methodology .....      | 15        |
| 1.3.7 Waterfall Methodology.....                | 15        |
| 1.3.8 When to Use Waterfall .....               | 16        |
| 1.3.9 Agile vs Waterfall Models.....            | 17        |
| 1.4 Technologies and Tools .....                | 18        |
| <b>Chapter Two: Analysis Phase .....</b>        | <b>19</b> |
| 2.1 Problem Definition .....                    | 20        |
| 2.2 Similar Platforms.....                      | 20        |
| Background .....                                | 20        |
| Review of Existing Systems .....                | 21        |
| Identified Gaps and Limitations .....           | 22        |
| Proposed Solution .....                         | 22        |
| 2.3 System Description.....                     | 22        |
| 2.3.1 Functional Requirements .....             | 23        |
| 2.3.2 Nonfunctional Requirements .....          | 24        |
| 2.3.3 System Context and Scope .....            | 24        |
| 2.4 System Modules .....                        | 25        |
| 2.4 System Models .....                         | 26        |
| 2.4.1 Entity Relationship Diagram (ERD) .....   | 26        |
| 2.4.2 Use Case Model .....                      | 27        |
| 2.4.3 Class Diagram.....                        | 29        |
| 2.4.4 Data Flow Diagram (DFD) .....             | 30        |
| 2.4.5 Sequence Diagram.....                     | 31        |
| <b>Chapter Three: System Design Phase .....</b> | <b>32</b> |
| 3.1 System Architecture .....                   | 33        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 3.1.1 Architectural Style.....                          | 33        |
| 3.1.2 High-Level System Architecture.....               | 33        |
| 3.1.3 Architecture Diagram .....                        | 34        |
| 3.1.4 Technology Stack.....                             | 35        |
| 3.1.5 Non-Functional Requirements in Architecture ..... | 35        |
| 3.1.6 Data Flow Overview .....                          | 36        |
| 3.2 Database Design (Schema) .....                      | 36        |
| <b>Chapter Four: Artificial Intelligence .....</b>      | <b>37</b> |
| 4.1 Introduction .....                                  | 38        |
| 4.2 Objectives of the AI Chatbot .....                  | 38        |
| 4.3 Technologies Used .....                             | 38        |
| 4.4 Chatbot System Architecture .....                   | 39        |
| 4.5 Features of the AI Chatbot .....                    | 39        |
| 4.6 Artificial Intelligence Techniques Used .....       | 39        |
| 4.6.1 Natural Language Processing (NLP).....            | 39        |
| 4.6.2 Intent Recognition .....                          | 39        |
| 4.6.3 Automated Response Generation .....               | 39        |
| 4.7 Advantages of Using AI in the System.....           | 40        |
| 4.8 Limitations of the Chatbot .....                    | 40        |
| 4.9 Future Improvements.....                            | 40        |
| 4.10 Conclusion .....                                   | 41        |
| 4.11 Screenshots.....                                   | 41        |
| <b>Chapter Five: Implementation .....</b>               | <b>44</b> |
| 5.1 Why We Chose Flutter .....                          | 45        |
| 5.2 Flutter Packages Used.....                          | 45        |
| 5.3 Why We Chose Node.js in Website .....               | 45        |
| 1. High Performance for Real-Time Apps .....            | 46        |
| 2. JavaScript Everywhere (Full-Stack Unity) .....       | 46        |
| 3. Massive Ecosystem (NPM) .....                        | 46        |
| 4. Microservices Friendly.....                          | 46        |
| 5. Cost-Effective Scaling.....                          | 46        |
| Summary: When to Use Node.js.....                       | 47        |
| 5.4 Website Packages Used .....                         | 47        |
| 1. Core Frameworks.....                                 | 47        |
| 2. Database & Data Management .....                     | 47        |
| 3. Security & Authentication .....                      | 48        |
| 5. Utility & DevOps .....                               | 48        |
| 5.5 Some Code from the Website Implementation .....     | 49        |
| 5.6 Some Code from Flutter .....                        | 52        |
| <b>Chapter Six: Testing.....</b>                        | <b>64</b> |

---

|                                                       |           |
|-------------------------------------------------------|-----------|
| 6.1 Introduction .....                                | 66        |
| 6.2 Project Description .....                         | 66        |
| 6.3 Testing Scope .....                               | 66        |
| 6.4 Testing Methods .....                             | 67        |
| 6.4.1 Integration Testing .....                       | 67        |
| 6.4.2 System Testing .....                            | 67        |
| 6.4.3 User Acceptance Testing .....                   | 67        |
| 6.5 Tools and Frameworks .....                        | 67        |
| 6.6 Success Criteria .....                            | 68        |
| 6.7 Conclusion .....                                  | 68        |
| 6.8 Performance Testing .....                         | 68        |
| <b>Chapter Seven: Conclusion .....</b>                | <b>69</b> |
| 7.1 Summary of Achieved Results .....                 | 70        |
| 7.2 Possible Extensions and Further Development ..... | 71        |

---

## Table of Figures

Figure 1: SDLC Iterative Model Diagram

Figure 2: Technologies and Tools Screenshot

Figure 3: Technologies Logo

Figure 4: Entity Relationship Diagram (ERD)

Figure 5: Use Case Diagram

Figure 6: Use Case Diagram (Extended)

Figure 7: Use Case Diagram (Admin View)

Figure 8: Class Diagram

Figure 9: Data Flow Diagram (DFD)

Figure 10: Sequence Diagram

Figure 11: High-Level System Architecture Diagram

Figure 12: Database Schema Design

Figure 13: AI Chatbot Screenshot 1

Figure 14: AI Chatbot Screenshot 2

Figure 15: AI Chatbot Screenshot 3

Figure 16: Website Code - DB Table 1

Figure 17: Website Code - DB Tables 2

Figure 18: Website Code - Admin Appointment

Figure 19: Website Code - VS Code View

Figure 20: patient Module dashboard

Figure 21: patient appointment

Figure 22: Patient Prescriptions Module

Figure 23: Flutter Code - Main Module

Figure 24: Flutter Code - Additional View

Figure 25: Flutter Code - Create Account

Figure 26: Flutter Code - Admin Code Detail Create Account (Part 1)

Figure 27: Flutter Code - Create Account (Part 2)

Figure 28: Website – Earning Overview

Figure 29: Website – Patient of Doctor

Figure 30: MobileApp– Patient Dashboard

Figure 31: MobileApp– Find Doctor

Figure 32: MobileApp– Patient's Prescription

Figure 33: MobileApp– Patient message doctor

Figure 34: MobileApp– Patient profile

Figure 35: MobileApp– Chat-Bot

Figure 36: MobileApp– Patient ask Ai assistant

---

## *Chapter one : Introduction*

---

## Chapter One: Introduction

### 1.1 Introduction

Nowadays many systems have been developed to make life easier. In this project, the aim is to serve both patients and doctors through a dual-role application, enabling users to interact as either a doctor or a patient. This distinction creates meaningful value for a wide range of users including healthcare professionals.

Communication between doctors and patients becomes simpler, allowing each party to access relevant information about the other with ease. The platform manages doctor appointments to streamline the reservation process.

Furthermore, the application provides significant benefits for elderly users and those suffering from contagious diseases by facilitating remote access to medical care and communication with healthcare professionals.

### 1.2 Project Objectives

- Replacing the manual work involved in managing clinics with an automated system.
- Reducing the time, cost, and effort required to complete any process.
- Facilitating the retrieval of the patient's medical history and any information the doctor needs.
- Allowing patients to choose the most suitable doctor based on cost and rating.
- Supporting different age groups, especially elderly people.

## 1.3 Project Development Methodology

### 1.3.1 Software Development Life Cycle

Software Development Life Cycle (SDLC) is a structured process that defines the phases involved in planning, creating, testing, deploying, and maintaining a software application. It helps teams deliver high-quality software on time, within budget, and with fewer errors.

#### Main Phases of SDLC

| # | Name                  | Main Activities                                                           | Key Deliverables                         |
|---|-----------------------|---------------------------------------------------------------------------|------------------------------------------|
| 1 | Planning              | Requirement gathering, feasibility study, project scope                   | Project plan, Feasibility report         |
| 2 | Requirements Analysis | Detailed functional & non-functional requirements, stakeholder interviews | Software Requirement Specification (SRS) |
| 3 | Design                | System architecture, UI/UX design, database design                        | Design Document (HLD + LLD), Wireframes  |
| 4 | Development / Coding  | Actual programming/coding based on design documents                       | Source Code, Working Software            |
| 5 | Testing               | Unit, integration, system testing, UAT                                    | Test cases, Bug reports, Test summary    |
| 6 | Deployment            | Release the software to production environment                            | Live application, Release notes          |
| 7 | Maintenance           | Bug fixing, updates, enhancements, monitoring                             | Patches, New versions                    |

### 1.3.2 Popular SDLC Models

| Model          | Key Characteristics                               | Best For                           |
|----------------|---------------------------------------------------|------------------------------------|
| Waterfall      | Linear, sequential phases                         | Small, well-defined projects       |
| Agile          | Iterative, flexible, customer feedback            | Most modern web/mobile apps        |
| Scrum (Agile)  | 2–4 week sprints, daily standups                  | Teams that want fast delivery      |
| Kanban         | Visual workflow, continuous delivery              | Maintenance & support projects     |
| Spiral         | Risk-driven, combines iterative + waterfall       | Large, complex, high-risk projects |
| V-Model        | Extension of Waterfall with testing in each phase | Safety-critical systems            |
| DevOps / CI-CD | Continuous Integration & Deployment               | Cloud-native, frequent releases    |

### 1.3.3 Iterative Model

The Iterative model focuses on repeated cycles of prototyping, refinement, and testing. Each iteration builds upon the previous one, incorporating feedback and improvements. This methodology is beneficial when requirements are not well understood initially or are expected to change frequently.

### 1.3.4 Agile Methodology



*Figure 1: SDLC Iterative Model Diagram*

Agile is a flexible and incremental approach that emphasizes adaptability. It divides the work into manageable iterations called sprints, typically lasting two to four weeks. Agile promotes continuous collaboration between cross-functional teams, allowing for regular refinement and rapid development.

### 1.3.5 When to Use Agile

- Requirements are not well understood or expected to change frequently.
- Frequent feedback from stakeholders is essential.
- Rapid delivery of functional pieces is necessary.
- Risk tolerance is high and accommodates changes and uncertainties.

### 1.3.6 Why We Chose Agile Methodology

- **Evolving and Changing Requirements:** Healthcare users often refine their needs as they see working parts of the system.
- **Continuous User Feedback:** Regular input from doctors and patients was essential for usability improvements.
- **Early Delivery of Working Software:** Agile enabled incremental delivery of functional modules.
- **Better Risk Management:** Issues are identified and resolved early through frequent testing and sprint reviews.
- **Higher Flexibility:** Agile adapts to a dynamic healthcare environment with evolving features.
- **Improved Collaboration and Transparency:** Daily stand-ups and sprint reviews ensured strong collaboration.
- **Continuous Testing and Quality Improvement:** Testing throughout the cycle ensures higher software quality.

### 1.3.7 Waterfall Methodology

The Waterfall Model is a traditional, linear, and sequential approach to software development, introduced by Dr. Winston W. Royce in 1970. Each phase must be fully completed and approved before moving to the next. The process is highly structured and disciplined.

#### Phases of the Waterfall Model

- **Requirements Analysis:** All functional and non-functional requirements are gathered and documented.
- **System Design:** High-Level and Low-Level Designs are created, including system architecture and database design.
- **Implementation (Coding):** Actual source code is written based on the approved design documents.
- **Testing:** Complete system testing after the development phase, including unit, integration, system, and UAT.
- **Deployment:** Software is deployed to the production environment after successful testing.
- **Maintenance:** Post-deployment support, bug fixing, and system enhancements.

### 1.3.8 When to Use Waterfall

- Well-defined and stable requirements.
  - Regulatory and compliance-driven projects.
  - Safety-critical systems.
  - Small to medium-sized projects.
  - Projects with fixed budget and timeline.
  - Projects requiring extensive documentation.
-

### 1.3.9 Agile vs Waterfall Models

| Aspect               | Waterfall Model                        | Agile Model                                 |
|----------------------|----------------------------------------|---------------------------------------------|
| Approach             | Linear and Sequential                  | Iterative and Incremental                   |
| Flexibility          | Very Low – Requirements fixed early    | High – Changes welcomed late in development |
| Requirements         | Must be fully defined before starting  | Requirements evolve throughout              |
| Delivery Speed       | Slow – Working software at the end     | Fast – Delivered in short iterations        |
| Testing              | Only after development is complete     | Continuous throughout the project           |
| Customer Involvement | Low – Mainly at beginning and end      | High – Continuous feedback every sprint     |
| Documentation        | Heavy and comprehensive                | Lightweight and just enough                 |
| Risk Level           | High – Issues discovered late          | Low – Issues identified and fixed early     |
| Cost of Change       | Very High (especially in later phases) | Low – Changes implemented relatively easily |

## 1.4 Technologies and Tools

The system was developed using the following technologies and tools:



*Figure 2: Technologies and Tools Screenshot*

---

## Chapter Two: Analysis Phase

---

## Chapter Two: Analysis Phase

### 2.1 Problem Definition

- Manual management of clinic data leads to data loss, redundancy, and difficulty retrieving patient information.
- Patients often lack sufficient information about nearby clinics, services, or available doctors.
- Manual appointment booking wastes time and causes unnecessary delays for both patients and clinic staff.
- Many people, especially elderly patients, struggle to remember medication doses and scheduled appointments.
- The absence of an integrated system makes it difficult to organize medical records and provide a smooth healthcare experience.

### 2.2 Similar Platforms

#### Background

The healthcare industry has experienced a paradigm shift toward digitalization, driven by technological advancements and changing patient expectations. Traditional healthcare delivery models are increasingly being complemented or replaced by digital alternatives. The COVID-19 pandemic accelerated this transformation, demonstrating the viability and necessity of remote healthcare services.

## Review of Existing Systems

### Telemedicine Platforms

**Teladoc Health:** Offers 24/7 virtual consultations, mental health services, and chronic condition management. Strengths include extensive provider network and global presence; primarily US-focused with higher costs for uninsured patients.

**Doctor On Demand:** Provides video consultations for urgent care, mental health, and preventive care. User-friendly with same-day appointments, but with limited diagnostic capabilities.

**Practo:** Covers doctor search, appointment booking, online consultations, and health records, with a large provider database in India and Southeast Asia. Service quality remains inconsistent.

### Integrated Healthcare Platforms

**Zocdoc:** Specializes in doctor discovery, appointment scheduling, and patient reviews with comprehensive provider information, but lacks diagnostic integration.

**HealthTap:** Offers virtual consultations, AI symptom checker, and prescription management with 24/7 access to doctors, but has prescription fulfillment gaps.

**mfine:** Provides teleconsultations, medicine delivery, diagnostic tests, and health packages with an integrated approach, but has regional limitations.

## Identified Gaps and Limitations

- **Fragmented Services:** Most platforms excel in one area but lack seamless integration.
- **User Experience:** Disjointed experiences when transitioning between consultation, prescription, and pharmacy.
- **Data Silos:** Medical records, prescriptions, and diagnostic reports are not centralized.
- **Limited Follow-up:** Inadequate mechanisms for medication adherence monitoring.
- **Geographic Constraints:** Many platforms serve specific regions with limited scalability.
- **Communication Gaps:** Insufficient communication channels between doctors, pharmacists, and patients.

## Proposed Solution

This project aims to address these gaps by developing a comprehensive, integrated online medical platform that seamlessly combines telemedicine consultations, electronic prescription management, centralized electronic health records, real-time communication between all stakeholders, and transparent pricing and medication information.

## 2.3 System Description

The Online Medical Platform is a comprehensive web-based system designed to facilitate healthcare delivery through digital channels. The system connects multiple stakeholders and manages the entire care journey from consultation to prescribing medicine.

## 2.3.1 Functional Requirements

### User Registration and Authentication

- Patients and doctors can register using email/phone number.
- Doctor credentials (license number, specialization) must be verified before approval.
- Secure login with password reset and multi-factor authentication options.

### Appointment Management

- Patients can search doctors by specialization and view availability.
- Booking appointments with available time slots.
- Rescheduling and cancellation with appropriate policies.
- Automated reminders before scheduled appointments.

### Virtual Consultation

- Support for video, audio, and text-based consultations.
- Access to patient medical history during consultation.
- Document consultation notes and share files (reports, images).

### Prescription Management

- Electronic prescription creation with medication details (name, dosage, frequency, duration).
- Prescription validation and history maintenance.
- Secure access for patients and pharmacists.

### Health Records Management

- Electronic health records (EHR) for each patient.
- Document uploads (medical reports, prescriptions, images).
- Record organization by date and category with role-based access control.

### 2.3.2 Nonfunctional Requirements

- **Usability:** The system should be easy to use and provide a smooth user experience with an intuitive, clear, and accessible interface.
- **Security & Privacy:** The system must ensure authentication, authorization, and data encryption to maintain confidentiality and prevent unauthorized access.
- **Performance & Responsiveness:** Fast appointment booking, real-time notifications, quick access to medical records, and minimal loading times.
- **Compatibility & Accessibility:** Compatible across multiple platforms including web browsers and Android devices.
- **Scalability:** The system should scale efficiently to handle a growing number of patients, doctors, medical records, and transactions.

### 2.3.3 System Context and Scope

#### System Stakeholders — Primary Users

- **Patients:** Individuals seeking medical consultations, prescriptions, and diagnostic services.
- **Doctors:** Licensed medical practitioners providing virtual consultations and prescriptions.
- **Administrators:** System managers overseeing operations, compliance, and user management.

#### Secondary Stakeholders

- **Payment Gateways:** Third-party payment processors.
- **Regulatory Authorities:** Healthcare and pharmaceutical regulatory bodies.

## 2.4 System Modules

### Patient Module

- User registration and profile management.
- Doctor search, appointment booking, and virtual consultation interface.
- Prescription viewing, health records repository, and payment/billing.

### Doctor Module

- Professional profile and credential management.
- Availability and schedule management.
- Electronic prescription generation and patient medical history access.
- Revenue and analytics dashboard.

### Admin Module

- User management (verification, approval, suspension).
- System monitoring, analytics, and transaction oversight.
- Content management, compliance/audit reporting, and system configuration.

## 2.4 System Models

### 2.4.1 Entity Relationship Diagram (ERD)

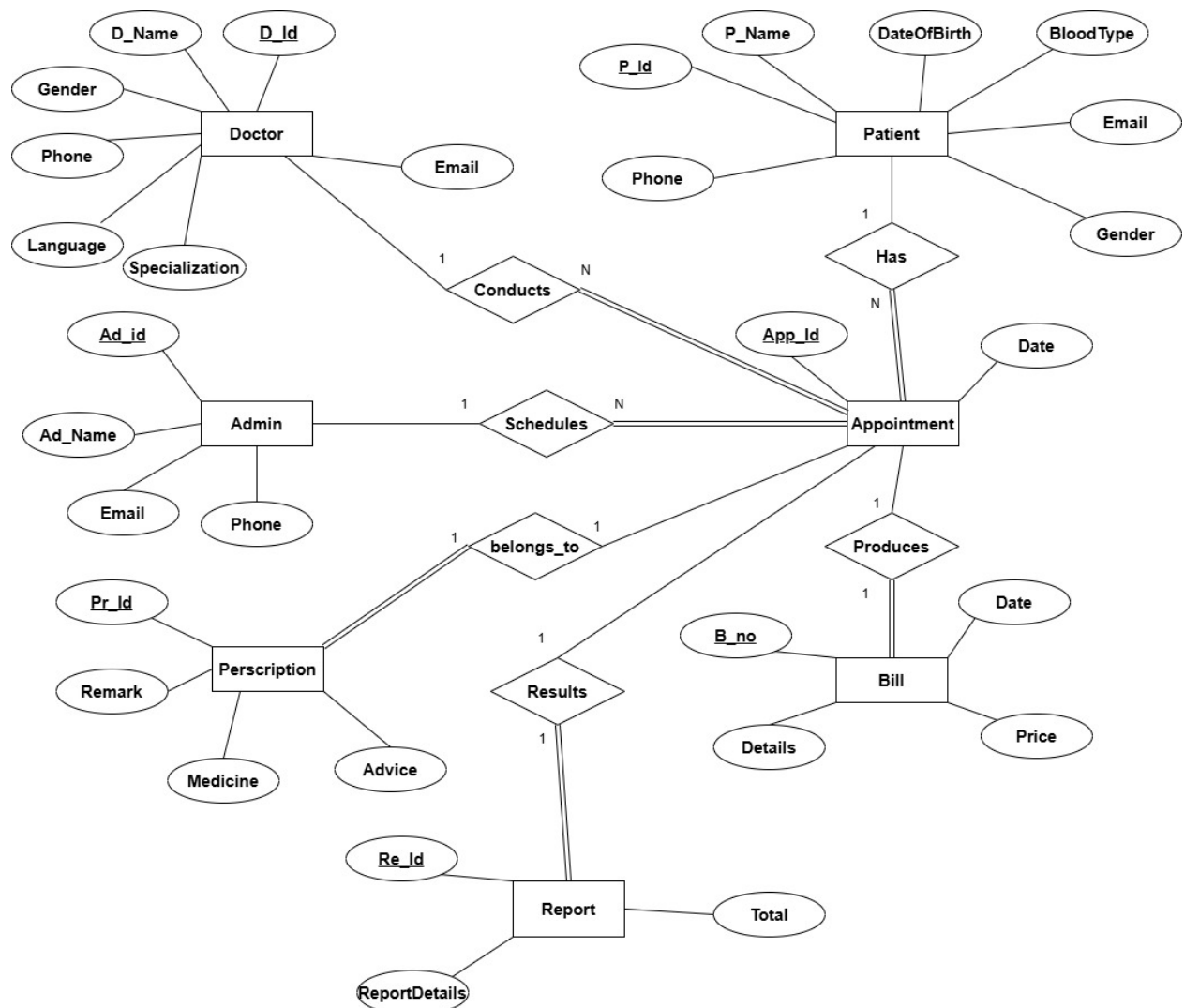


Figure 4: Entity Relationship Diagram (ERD)

## 2.4.2 Use Case Model

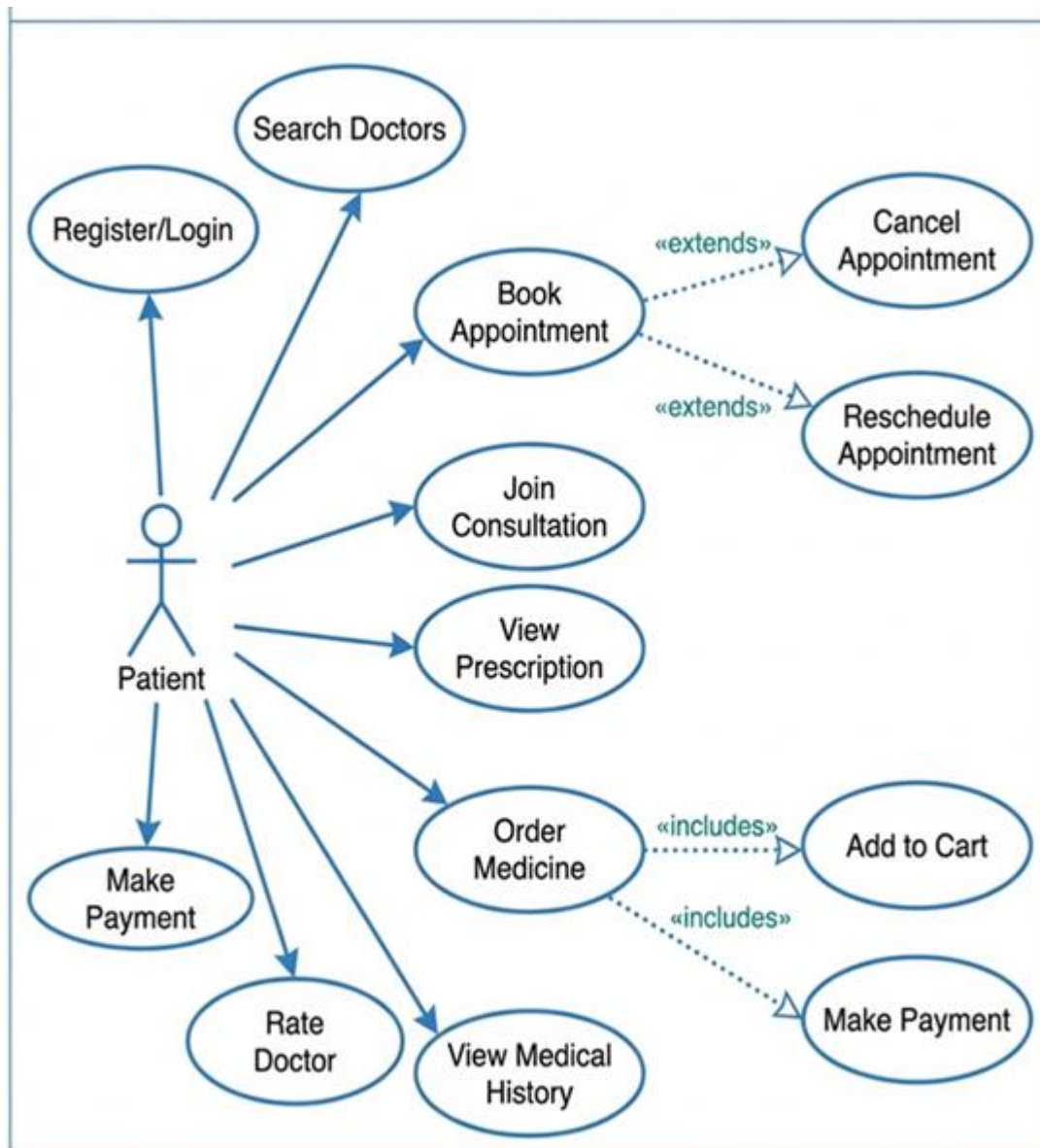


Figure 5: Use Case Diagram

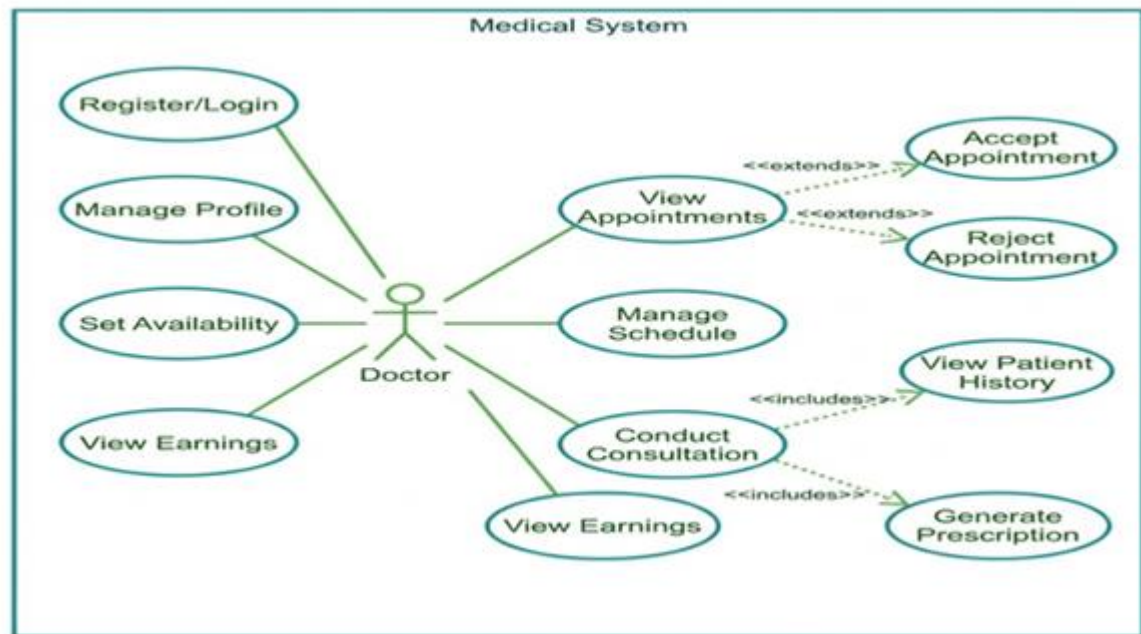


Figure 6: Use Case Diagram (Extended)

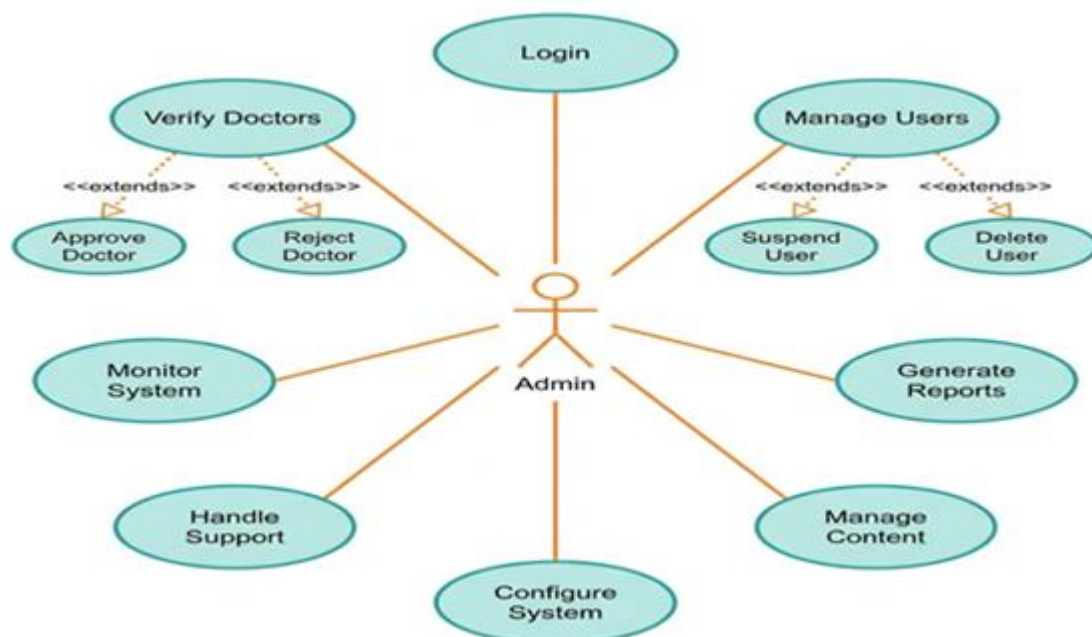


Figure 7: Use Case Diagram (Admin View)

## 2.4.3 Class Diagram

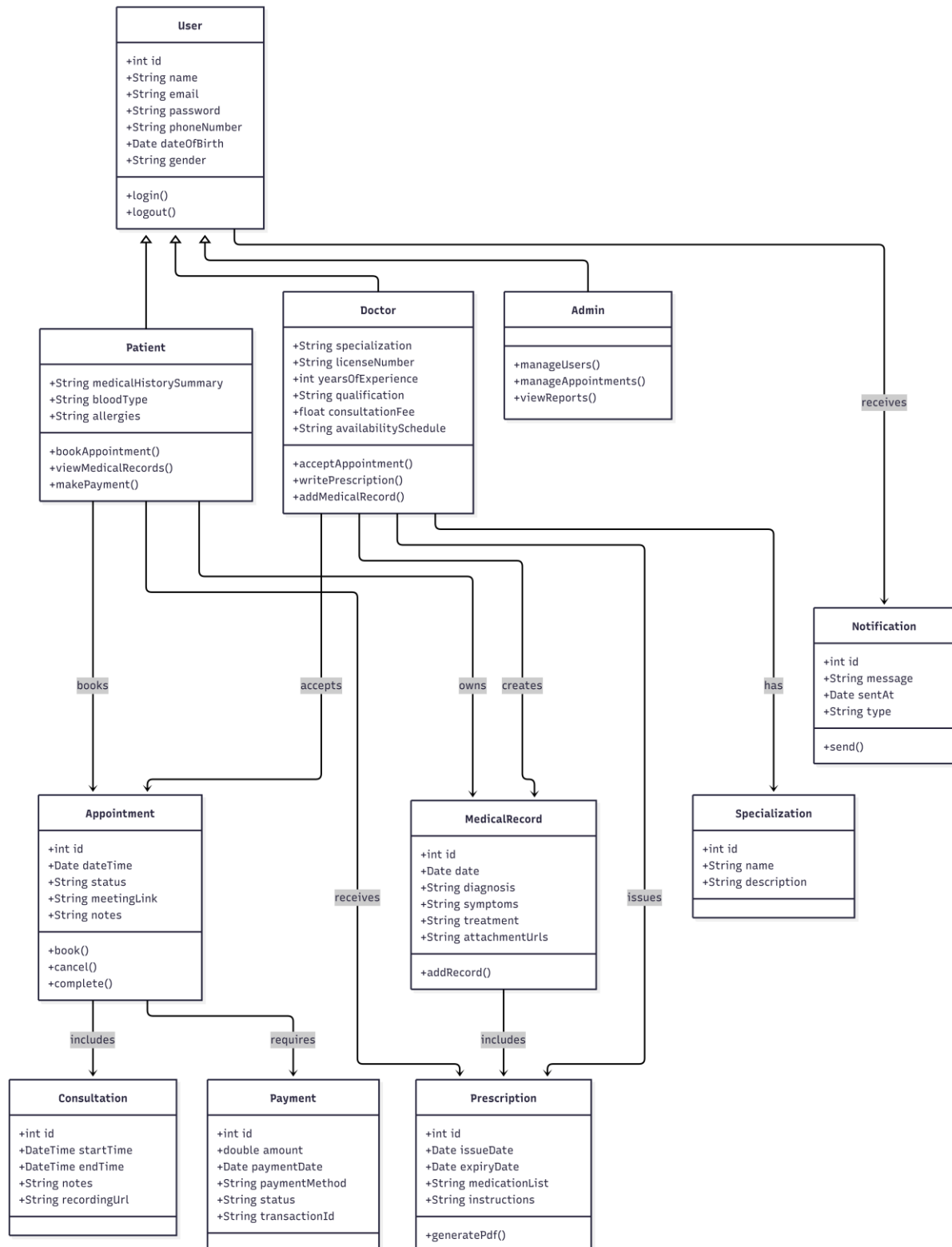


Figure 8: Class Diagram

## 2.4.4 Data Flow Diagram (DFD)

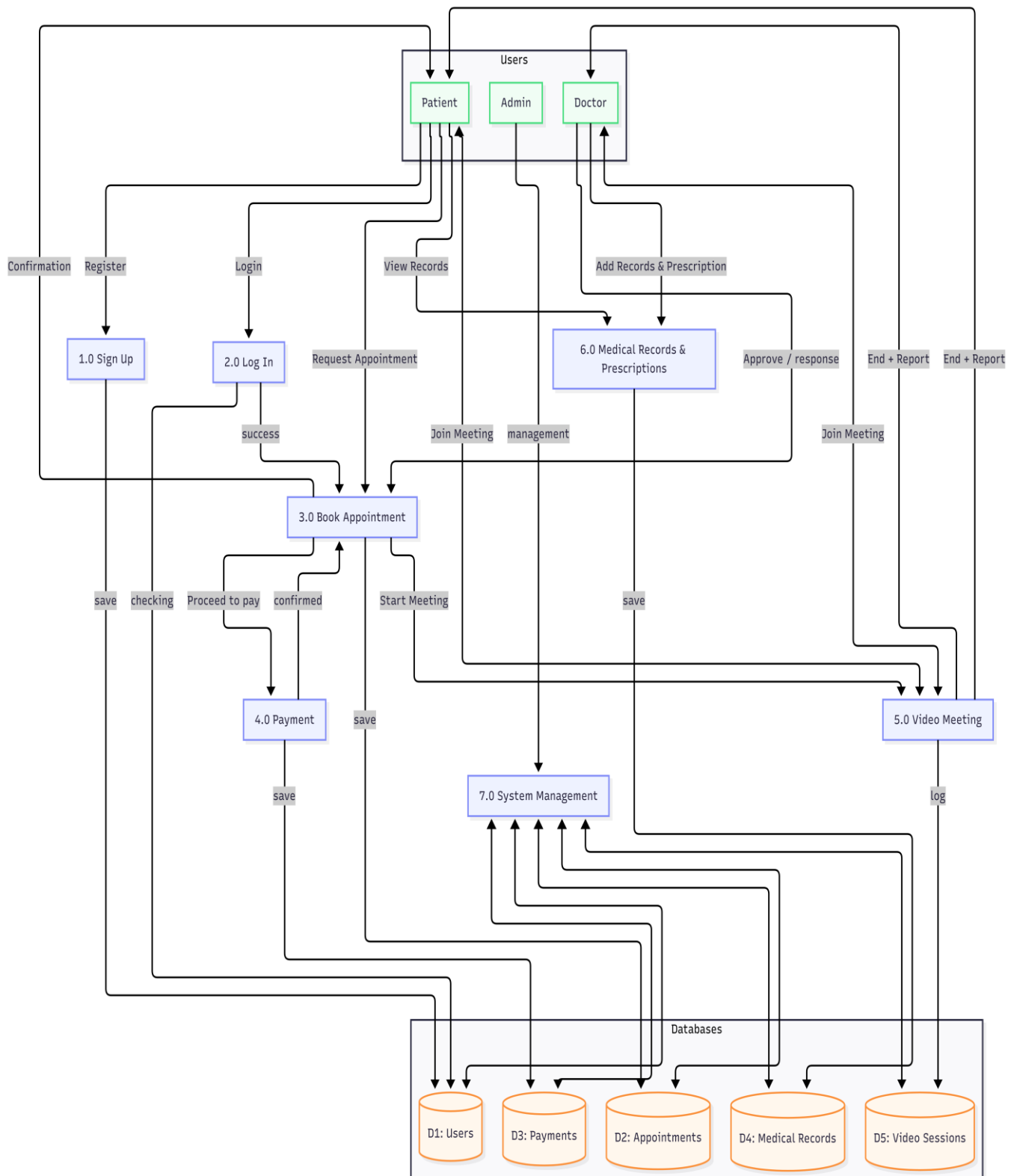


Figure 9: Data Flow Diagram (DFD)

## 2.4.5 Sequence Diagram

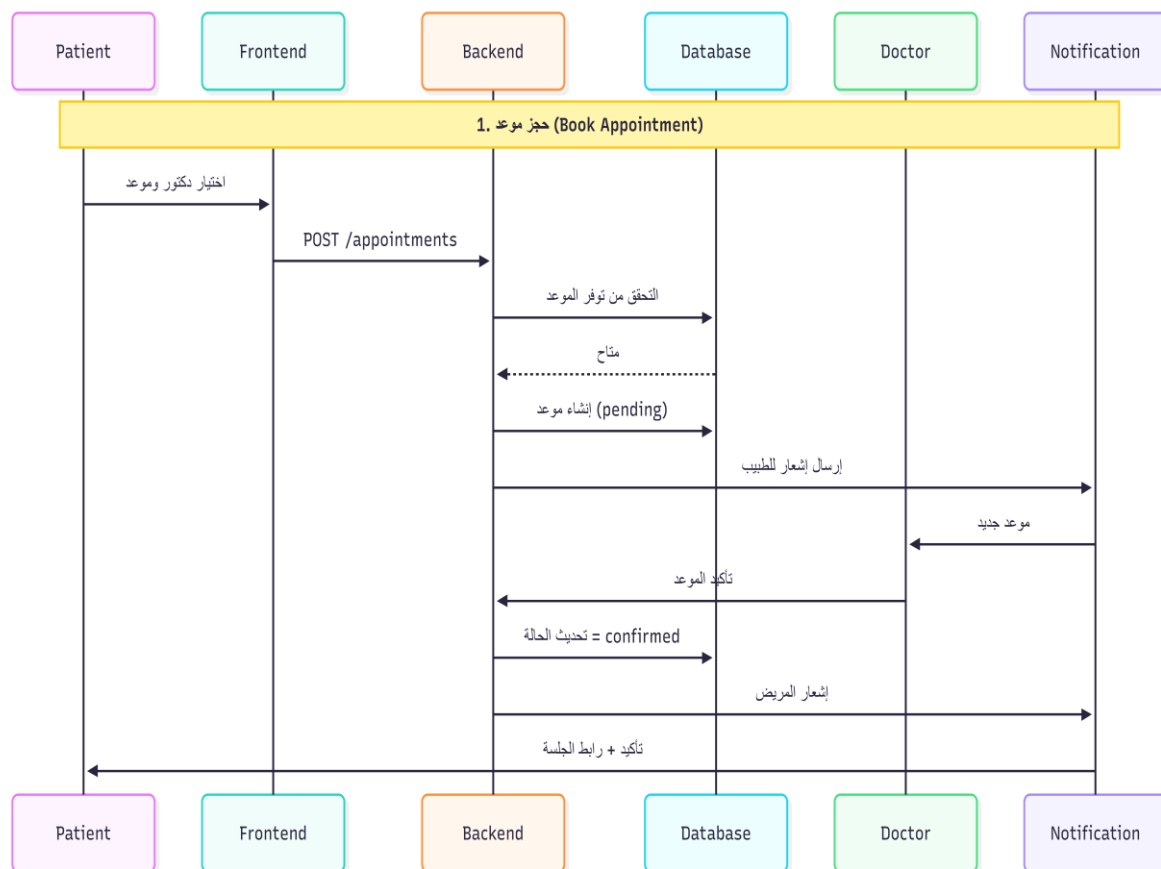


Figure 10: Sequence Diagram

---

## Chapter Three: System Design

---

## Chapter Three: System Design Phase

### 3.1 System Architecture

The proposed Online Medical System adopts a multi-tier client-server architecture (Layered Architecture) that ensures separation of concerns, scalability, maintainability, and high security — essential requirements in healthcare systems.

#### 3.1.1 Architectural Style

The system is built based on the Three-Tier Architecture with the possibility of expanding to Four-Tier Architecture in the future:

- Presentation Tier (Client Layer)
- Application Tier (Business Logic Layer)
- Data Tier (Database Layer)
- External Services Layer (integrations)

This architecture provides clear separation between user interface, business logic, and data storage, allowing each layer to be developed, tested, and scaled independently.

#### 3.1.2 High-Level System Architecture

- Frontend (Presentation Layer): Responsive web application supporting patients, doctors, and administrators through different dashboards.
- Backend (Application Layer): RESTful API handling all business logic, authentication, authorization, appointment scheduling, medical records, and prescriptions.
- Database Layer: Relational database management system designed to ensure data integrity, security, and quick retrieval of medical records.
- External Services & Integrations: Video Conferencing Service, Payment Gateway, Notification Service, File Storage Service, and PDF Generation Service.

### 3.1.3 Architecture Diagram

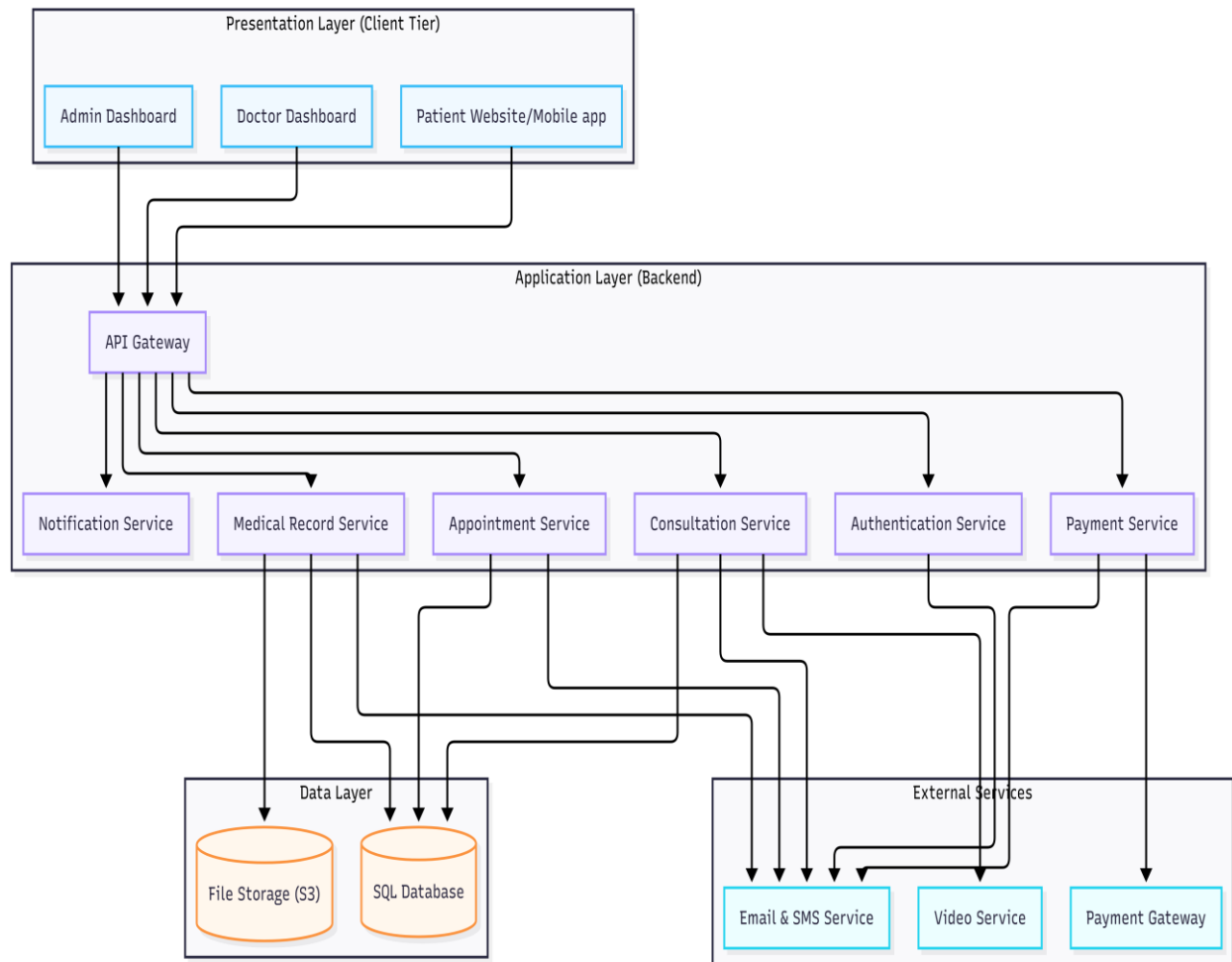


Figure 11: High-Level System Architecture Diagram

### 3.1.4 Technology Stack

| Layer          | Technology            | Justification                        |
|----------------|-----------------------|--------------------------------------|
| Frontend       | React.js              | Fast, responsive, component-based    |
| Backend        | Node.js               | High performance & rapid development |
| Database       | SQL                   | Strong relational support + security |
| Authentication | OAuth2 + bcrypt       | Secure user authentication           |
| Hosting        | AWS / Vercel + Render | Scalability & reliability            |
| File Storage   | AWS S3                | Secure storage of medical files      |

### 3.1.5 Non-Functional Requirements in Architecture

- **Security:** Role-Based Access Control (RBAC), HIPAA-inspired privacy standards, data encryption, audit logging.
- **Scalability:** Horizontal scaling of backend servers and database read replicas.
- **Availability:** 99.9% uptime target through load balancing.
- **Performance:** Fast response time (< 2 seconds) for critical operations.
- **Maintainability:** Clean code architecture and comprehensive documentation.

### 3.1.6 Data Flow Overview

- Patient registers and logs in → Frontend sends request to Backend.
- Backend validates request and interacts with Database.
- When booking an appointment → System checks doctor's availability in real-time.
- During consultation → Video stream goes through external Video Server while metadata is saved in the local database.
- After consultation → Doctor adds medical record and prescription → Patient receives notification and document.

## 3.2 Database Design (Schema)

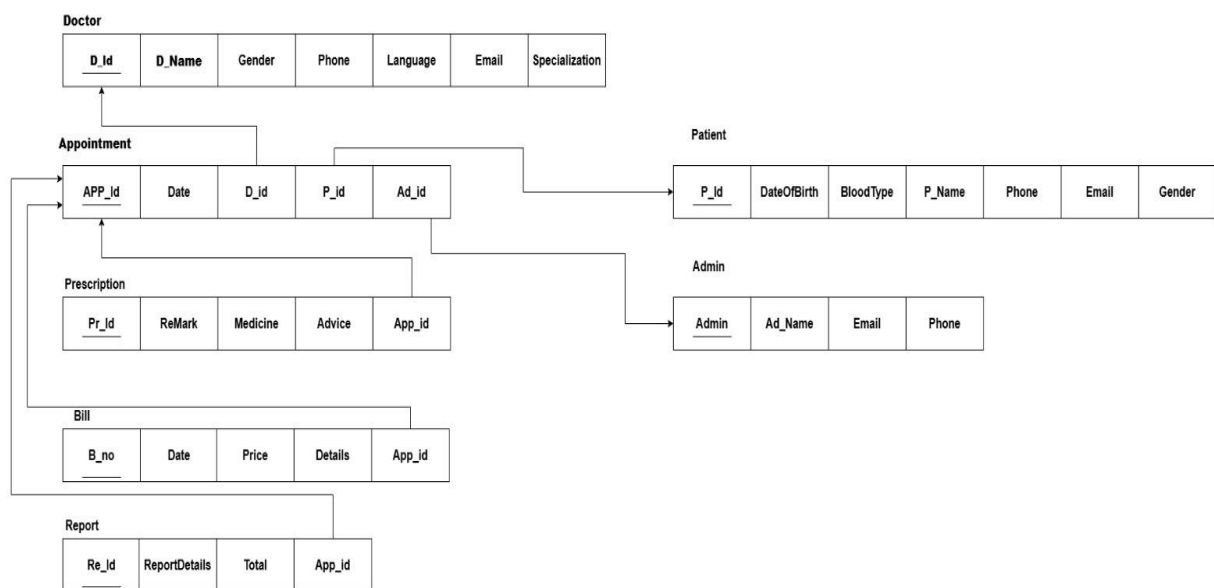


Figure 12: Database Schema Design

---

## Chapter Four : AI

---

## Chapter Four: Artificial Intelligence

### 4.1 Introduction

Artificial Intelligence (AI) has become an essential technology in modern healthcare systems. It helps improve healthcare services by providing intelligent solutions that enhance communication, save time, and increase system efficiency. In this project, an AI-powered chatbot was integrated into the Online Medical System to provide automated assistance and improve user interaction with the platform.

The chatbot enables users to communicate with the system easily and receive instant responses to common inquiries related to appointments, doctors, and medical services. This feature reduces the need for continuous human support and improves the overall user experience.

### 4.2 Objectives of the AI Chatbot

- To provide quick and accurate responses to users.
- To assist patients in navigating the medical system.
- To reduce the workload on administrative staff.
- To improve communication between patients and the system.
- To enhance user satisfaction and accessibility.

### 4.3 Technologies Used

- Programming Languages: Python / JavaScript
- Frontend Technologies: HTML, CSS, JavaScript
- Backend Technologies: Node.js / Django / Flask
- Artificial Intelligence APIs
- Natural Language Processing (NLP)
- Database Management Systems

## 4.4 Chatbot System Architecture

The chatbot architecture consists of several stages that allow effective interaction between the user and the system:

- The user enters a message through the chatbot interface.
- The message is sent to the backend server.
- The AI module processes and analyzes the request.
- The chatbot determines the user's intent.
- A suitable response is generated and returned to the user.
- The response is displayed instantly on the interface.

## 4.5 Features of the AI Chatbot

- 24/7 availability.
- Instant response generation.
- User-friendly interaction.
- Appointment assistance and guidance.
- Answering common medical inquiries.
- Supporting multiple users simultaneously.

## 4.6 Artificial Intelligence Techniques Used

### 4.6.1 Natural Language Processing (NLP)

NLP is used to help the chatbot understand and analyze user input written in natural language.

### 4.6.2 Intent Recognition

This technique allows the chatbot to identify the purpose of the user's message and provide an appropriate response.

### 4.6.3 Automated Response Generation

The chatbot generates automated responses based on the analyzed input and predefined logic.

## 4.7 Advantages of Using AI in the System

- Reducing response time.
- Enhancing user experience.
- Saving time and effort.
- Providing continuous support services.
- Increasing system efficiency and reliability.
- Reducing repetitive administrative tasks.

## 4.8 Limitations of the Chatbot

- It may not fully understand complex medical cases.
- Internet access is required for operation.
- Response quality depends on the AI model and training data.
- The chatbot cannot replace professional medical consultation.

## 4.9 Future Improvements

- Voice-based interaction support.
- Multilingual communication support.
- Integration with patient medical records.
- More advanced AI models for accurate responses.
- Smart medical recommendation systems.
- Mobile application integration.

## 4.10 Conclusion

The AI chatbot added significant value to the Online Medical System by improving communication and providing intelligent assistance to users. It enhanced the overall efficiency of the system and simplified the interaction process for patients. The integration of Artificial Intelligence demonstrates the potential of modern technologies in developing smarter and more effective healthcare solutions.

## 4.11 Screenshots



Figure 13: AI Chatbot Screenshot 1



Figure 14: AI Chatbot Screenshot 2



Figure 15: AI Chatbot Screenshot 3

---

## Chapter Five :Implementation

---

## Chapter Five: Implementation

### 5.1 Why We Chose Flutter

- **Single Codebase:** Developing for iOS, Android, and Web simultaneously from a unified code repository, which significantly reduces maintenance and resource costs.
- **Hot Reload:** A game-changing feature that allows for instant iteration, letting developers see UI changes in real-time without restarting the app.
- **Performance:** Flutter uses the Skia graphics engine to provide smooth, high-frame-rate (60–120fps) animations and native-level responsiveness.

### 5.2 Flutter Packages Used

- **Dio/Http:** For robust networking and asynchronous API communication.
- **Provider/Riverpod:** To manage application state efficiently across various screens.
- **URL Launcher:** Enabling the application to interact with external browser links and device features.
- **Shared Preferences:** For persisting local data like user settings and simple session storage.

### 5.3 Why We Chose Node.js in Website

Choosing Node.js for a web project is about solving specific architectural problems. The team prioritized speed, scalability, and developer efficiency.

### 5.3.1 High Performance for Real-Time Apps

Node.js is built on Google Chrome's V8 engine, which compiles JavaScript directly into machine code. Its "Non-blocking I/O" model allows it to handle thousands of concurrent connections without slowing down.

### 5.3.2 JavaScript Everywhere (Full-Stack Unity)

Using JavaScript for both the Frontend (React, Vue, Angular) and the Backend (Node.js) creates a seamless workflow. Code can be shared across the entire stack, and team efficiency increases as frontend developers can contribute to backend tasks.

### 5.3.3 Massive Ecosystem (NPM)

Node.js hosts NPM (Node Package Manager), the largest ecosystem of open-source libraries in the world. Instead of building complex features from scratch, you can pull in trusted, community-vetted packages.

### 5.3.4 Microservices Friendly

Because Node.js is lightweight, it's perfect for breaking a website into smaller, independent services that communicate via APIs. You can scale only the parts of the site experiencing heavy traffic without scaling the entire application.

### 5.3.5 Cost-Effective Scaling

Because Node.js is efficient with CPU and RAM, you can often handle the same traffic as a Java or PHP server using fewer server resources, directly translating to lower cloud hosting bills.

## Summary: When to Use Node.js

| Feature        | Why it Wins                                                        |
|----------------|--------------------------------------------------------------------|
| I/O Intensive  | Great for apps that talk to databases or APIs constantly.          |
| Data Streaming | Excellent for processing files or video while they upload.         |
| JSON Support   | Native support for JSON makes it the king of building APIs.        |
| Prototyping    | Extremely fast to set up and get a Minimum Viable Product running. |

## 5.4 Website Packages Used

### 5.4.1 Core Frameworks

- Express.js: The most popular, minimalist framework handling everything from simple web pages to complex APIs.
- NestJS: The go-to for enterprise apps using TypeScript with a strict, organized structure.
- Fastify: Gaining traction for being the fastest framework available with very low overhead.

### 5.4.2 Database & Data Management

- Prisma: A modern "next-generation" ORM making database queries feel like standard JavaScript.
- Mongoose: The standard for MongoDB, allowing you to define exactly what your data documents should look like.
- Sequelize: A veteran package for SQL databases (MySQL, PostgreSQL, SQLite).

### 5.4.3 Security & Authentication

- **Passport.js:** The "Swiss Army Knife" of authentication supporting over 500 login methods.
- **Helmet:** A vital must-have that protects your site by setting security-related HTTP headers.
- **jsonwebtoken (JWT):** Used to securely transmit information between parties, usually for maintaining login sessions.

| Category     | Package   | Purpose                                                          |
|--------------|-----------|------------------------------------------------------------------|
| Real-Time    | Socket.io | Enables live chat, instant notifications, and real-time updates. |
| File Uploads | Multer    | The standard middleware for handling image and file uploads.     |
| Validation   | Zod / Joi | Ensures that the data a user types into a form is valid.         |
| API Requests | Axios     | Used when your server needs to call another API.                 |

### 5.4.5 Utility & DevOps

- **Dotenv:** Loads environment variables from a .env file so they aren't exposed in code.
- **Winston / Morgan:** Record everything that happens on the server (logs).
- **PM2:** A process manager that ensures your website stays online and auto-restarts on crashes.

## 5.5 Some Code from the Website Implementation

```
-- =====
-- 3. DOCTORS TABLE
-- =====
CREATE TABLE doctors (
  id                INT IDENTITY(1,1) PRIMARY KEY,
  user_id           INT NOT NULL FOREIGN KEY REFERENCES users(id),
  specialty_id      INT NULL FOREIGN KEY REFERENCES specialties(id),
  bio               NVARCHAR(MAX) NULL,
  experience_years   INT DEFAULT 0,
  consultation_fee   DECIMAL(10,2) DEFAULT 0,
  clinic_name       NVARCHAR(255) NULL,
  clinic_address     NVARCHAR(500) NULL,
  available_days     NVARCHAR(100) DEFAULT 'Sat,Sun,Mon,Tue,Wed',
  available_from     NVARCHAR(10) DEFAULT '09:00',
  available_to       NVARCHAR(10) DEFAULT '17:00',
  rating            DECIMAL(3,2) DEFAULT 0,
  total_reviews     INT DEFAULT 0,
  status            NVARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'approved', 'suspended')),
  license_number     NVARCHAR(100) NULL,
  education          NVARCHAR(500) NULL,
  id_document        NVARCHAR(MAX) NULL,
  created_at        DATETIME2 DEFAULT GETDATE()
);
GO

-- =====
-- 4. PATIENTS TABLE
-- =====
CREATE TABLE patients (
  id                INT IDENTITY(1,1) PRIMARY KEY,
  user_id           INT NOT NULL FOREIGN KEY REFERENCES users(id),
  date_of_birth     DATE NULL,
  blood_type        NVARCHAR(5) NULL,
  height_cm         DECIMAL(5,1) NULL,
  weight_kg         DECIMAL(5,1) NULL,
  allergies          NVARCHAR(MAX) NULL,
  chronic_conditions NVARCHAR(MAX) NULL,
  emergency_contact_name NVARCHAR(255) NULL,
  emergency_contact_phone NVARCHAR(20) NULL,
  created_at        DATETIME2 DEFAULT GETDATE()
);
GO

-- =====
-- 5. APPOINTMENTS TABLE
-- =====
```

Figure 16: Website Code - DB Table 1

```
-- =====
-- 5. APPOINTMENTS TABLE
-- =====
CREATE TABLE appointments (
  id                INT IDENTITY(1,1) PRIMARY KEY,
  patient_id        INT NOT NULL FOREIGN KEY REFERENCES patients(id),
  doctor_id         INT NOT NULL FOREIGN KEY REFERENCES doctors(id),
  appointment_date   DATE NOT NULL,
  appointment_time   NVARCHAR(10) NOT NULL,
  type              NVARCHAR(20) DEFAULT 'in-person' CHECK (type IN ('in-person', 'video')),
  status            NVARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'completed', 'cancelled')),
  reason            NVARCHAR(MAX) NULL,
  notes             NVARCHAR(MAX) NULL,
  fee               DECIMAL(10,2) DEFAULT 0,
  created_at        DATETIME2 DEFAULT GETDATE()
);
GO

-- =====
-- 6. PAYMENTS TABLE
-- =====
CREATE TABLE payments (
  id                INT IDENTITY(1,1) PRIMARY KEY,
  appointment_id     INT NOT NULL FOREIGN KEY REFERENCES appointments(id),
  patient_id        INT NOT NULL FOREIGN KEY REFERENCES patients(id),
  doctor_id         INT NOT NULL FOREIGN KEY REFERENCES doctors(id),
  amount            DECIMAL(10,2) NOT NULL,
  payment_method     NVARCHAR(50) DEFAULT 'card',
  card_last4        NVARCHAR(4) NULL,
  transaction_id     NVARCHAR(100) NULL,
  status            NVARCHAR(20) DEFAULT 'completed' CHECK (status IN ('pending', 'completed', 'refunded')),
  created_at        DATETIME2 DEFAULT GETDATE()
);
GO

-- =====
-- 7. PRESCRIPTIONS TABLE
-- =====
CREATE TABLE prescriptions (
  id                INT IDENTITY(1,1) PRIMARY KEY,
  appointment_id     INT NOT NULL FOREIGN KEY REFERENCES appointments(id),
  patient_id        INT NOT NULL FOREIGN KEY REFERENCES patients(id),
  doctor_id         INT NOT NULL FOREIGN KEY REFERENCES doctors(id),
  medications        NVARCHAR(MAX) NULL, -- JSON string
  diagnosis          NVARCHAR(MAX) NULL,
  notes             NVARCHAR(MAX) NULL,
  created_at        DATETIME2 DEFAULT GETDATE()
);
GO
```

Figure 17: Website Code - DB Tables 2

```

import React, { useState, useEffect } from 'react';
import { api } from '../utils/api.js';

const C = { primary: '#3b82f6', dark: '#0f172a', gray: '#64748b', green: '#10b981', orange: '#f59e0b', red: '#ef4444' };
const STATUS_COLORS = { pending: { bg: '#fef9c3', c: '#854d0e' }, confirmed: { bg: '#dbeafe', c: '#1d4ed8' }, completed: { bg: '#d1fae5', c: '#065f46' }, cancelled: { bg: '#fee2e2', c: '#991b1b' } };

export default function AdminAppointments() {
  const [appointments, setAppointments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [filter, setFilter] = useState('all');

  useEffect(() => {
    api.getAllAppointments().then(data => { setAppointments(data); setLoading(false); }).catch(() => setLoading(false));
  }, []);

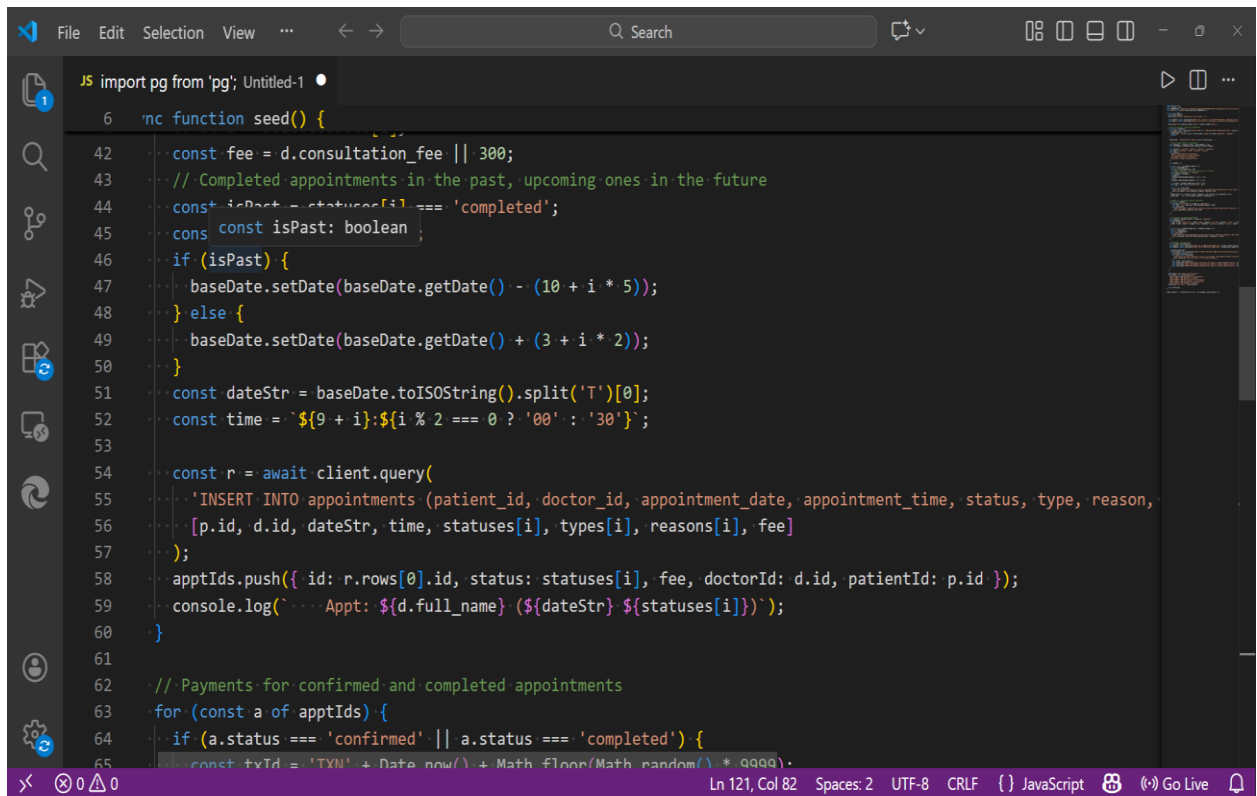
  const filtered = appointments.filter(a => {
    const matchSearch = !search || a.patient_name?.toLowerCase().includes(search.toLowerCase()) ||
    a.doctor_name?.toLowerCase().includes(search.toLowerCase());
    const matchFilter = filter === 'all' || a.status === filter;
    return matchSearch && matchFilter;
  });

  const counts = { all: appointments.length };
  ['pending', 'confirmed', 'completed', 'cancelled'].forEach(s => { counts[s] = appointments.filter(a => a.status === s).length; });

  return (
    <div className="fade" style={{ padding: 32 }}>
      <div style={{ marginBottom: 24 }}>
        <h1 style={{ fontSize: 26, fontWeight: 800, color: C.dark }}>All Appointments</h1>
      </div>
      <div style={{ color: C.gray, fontSize: 15, marginTop: 4 }}>{appointments.length} total appointments</div>
    </div>
  );
}

```

*Figure 18: Website Code - Admin Appointment*



The image shows a screenshot of the Visual Studio Code (VS Code) editor interface. The editor is displaying a JavaScript file named 'Untitled-1'. The code is written in a dark theme and includes comments in green. The code defines a 'seed' function that interacts with a database to insert appointments. The function uses a 'pg' module for database operations and a 'client' object for queries. It includes logic to determine if an appointment is in the past or future, set dates accordingly, and insert data into the 'appointments' table. The status of the appointment is set to 'completed' for past appointments and 'confirmed' for future ones. The code also includes a loop to generate a series of appointments and a final console log statement.

```
JS import pg from 'pg'; Untitled-1
6  nc function seed() {
42  const fee = d.consultation_fee || 300;
43  // Completed appointments in the past, upcoming ones in the future
44  const isPast = statuses[i] === 'completed';
45  const isPast: boolean;
46  if (isPast) {
47    baseDate.setDate(baseDate.getDate() - (10 + i * 5));
48  } else {
49    baseDate.setDate(baseDate.getDate() + (3 + i * 2));
50  }
51  const dateStr = baseDate.toISOString().split('T')[0];
52  const time = `${9 + i}:${i % 2 === 0 ? '00' : '30'}`;
53
54  const r = await client.query(
55    'INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_time, status, type, reason,
56    [p.id, d.id, dateStr, time, statuses[i], types[i], reasons[i], fee]
57  );
58  apptIds.push({ id: r.rows[0].id, status: statuses[i], fee, doctorId: d.id, patientId: p.id });
59  console.log(`Appt: ${d.full_name} (${dateStr} ${statuses[i]})`);
60  }
61
62  // Payments for confirmed and completed appointments
63  for (const a of apptIds) {
64    if (a.status === 'confirmed' || a.status === 'completed') {
65      const txId = 'TXN' + Date.now() + Math.floor(Math.random() * 9999);
```

Figure 19: Website Code - VS Code View

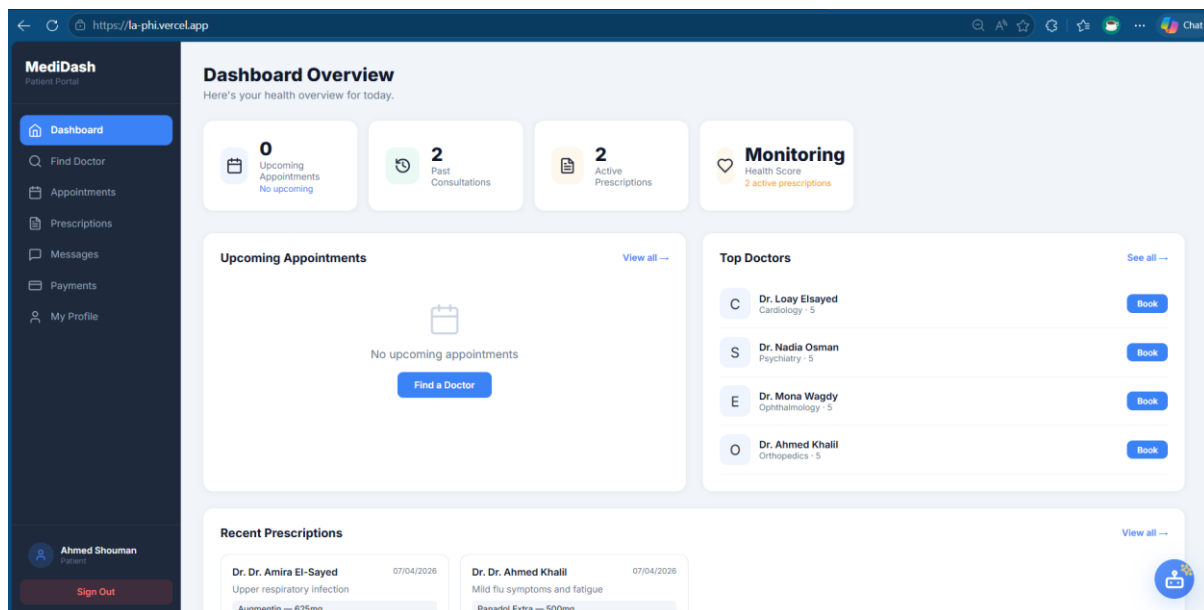


Figure 20: patient Module dashboard

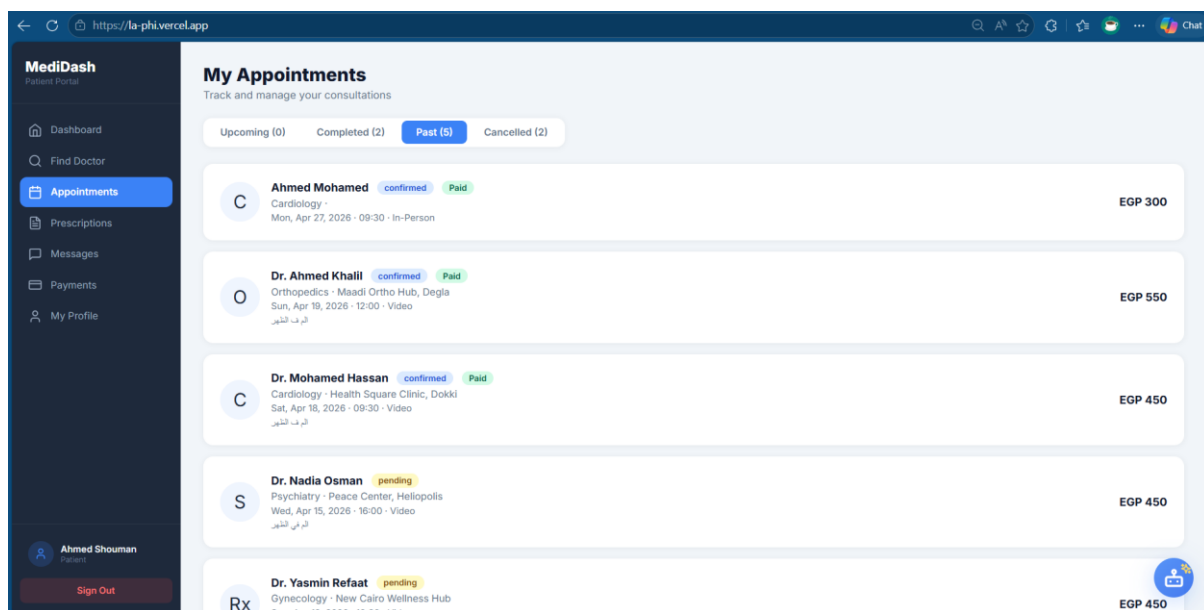


Figure 21: patient appointment

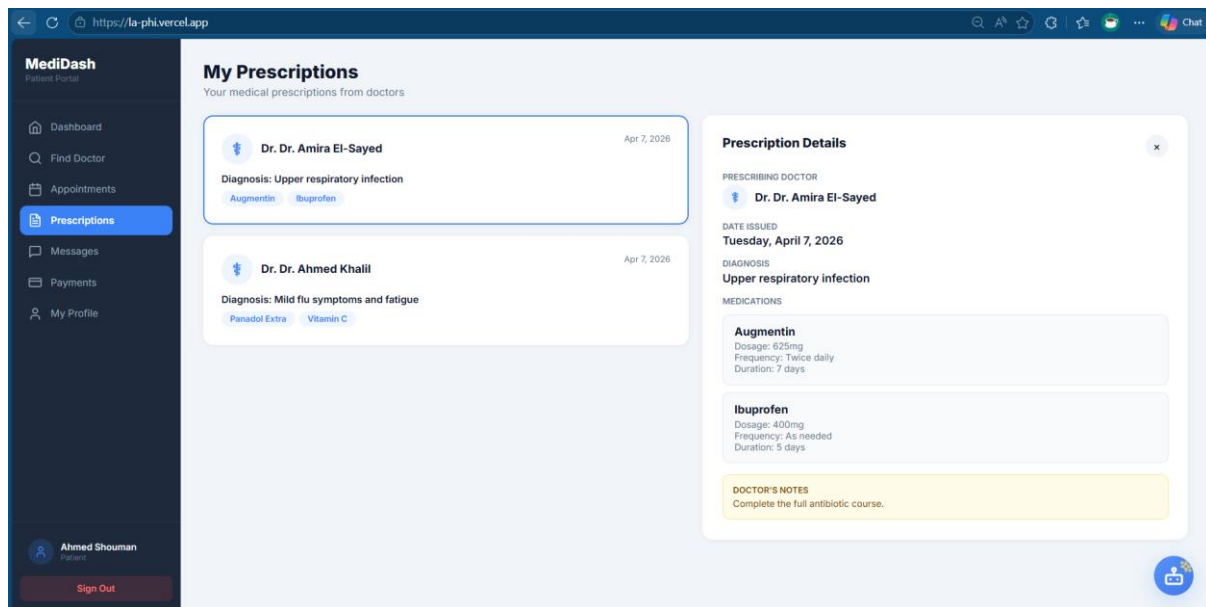


Figure 22: Patient Prescriptions Module

## 5.6 Some Code from Flutter

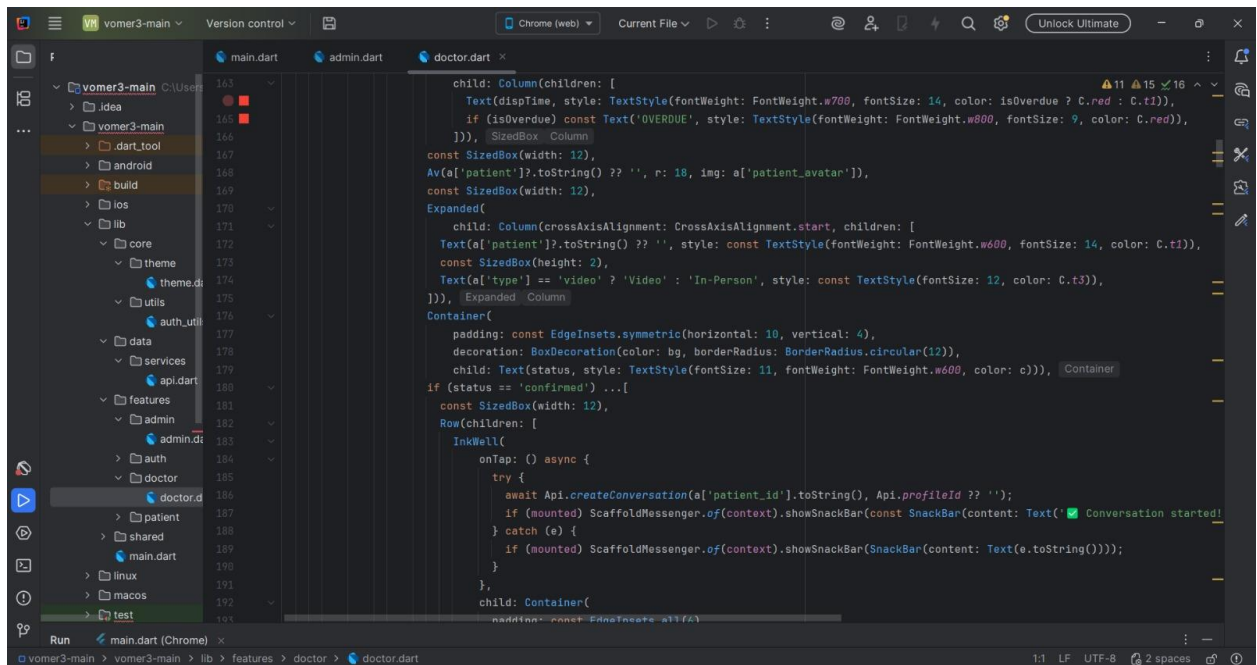


Figure 23: Flutter Code - Main Module

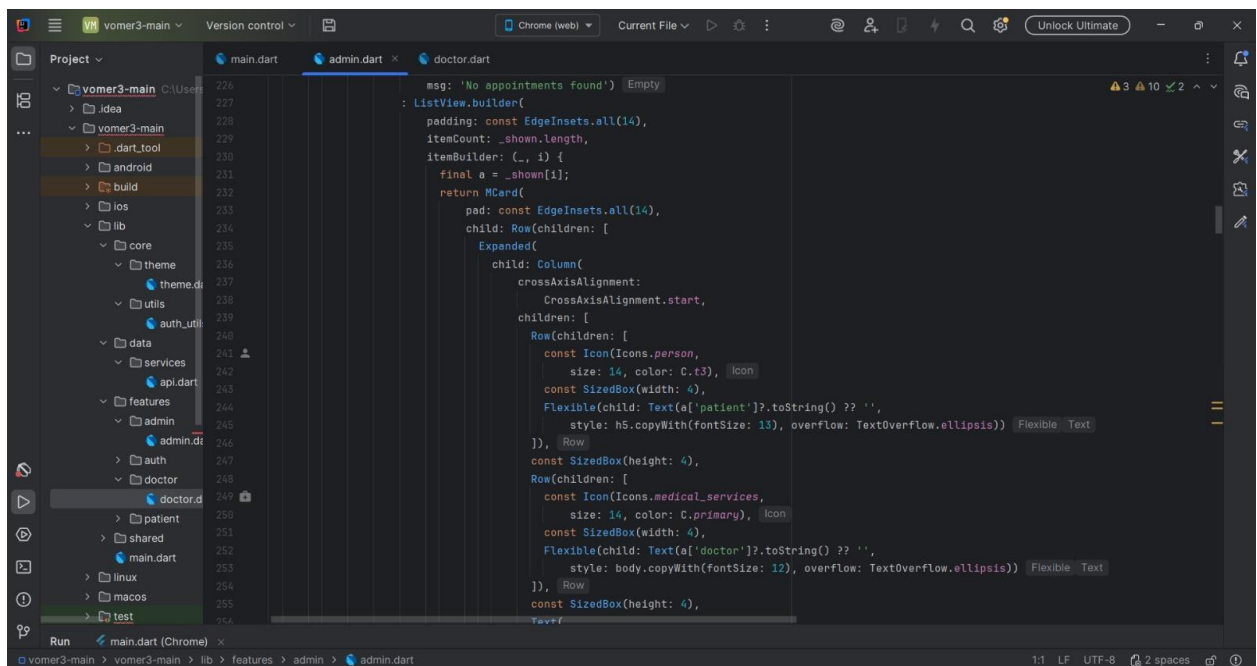


Figure 24: Flutter Code - Additional View

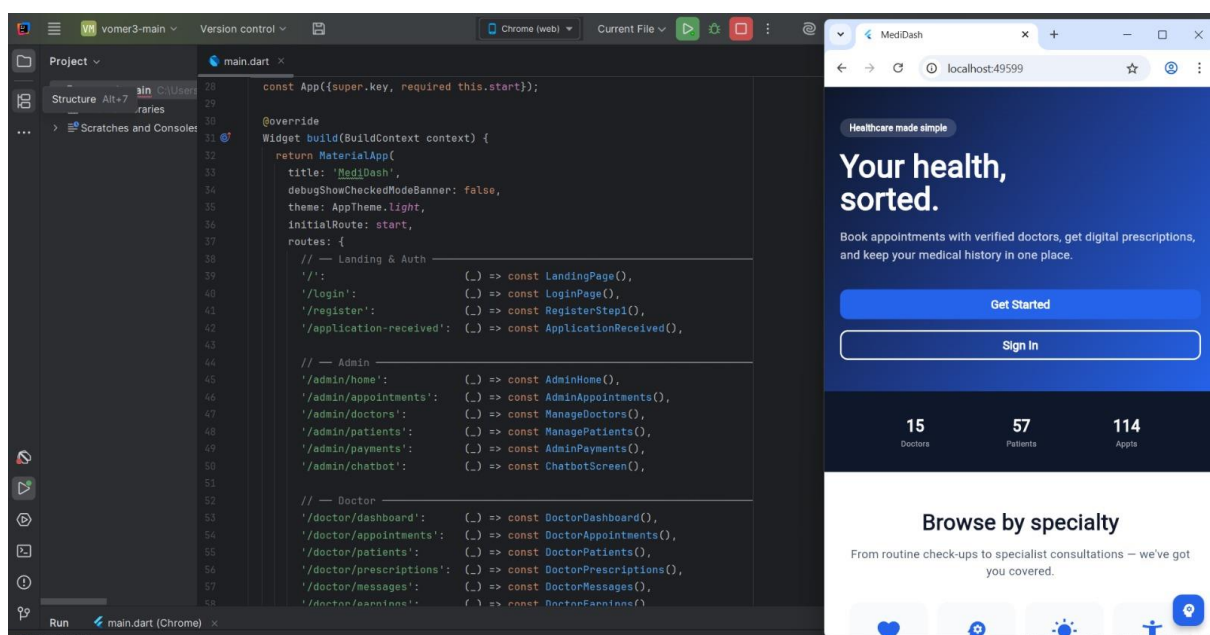


Figure 25: Flutter Code - Create Account

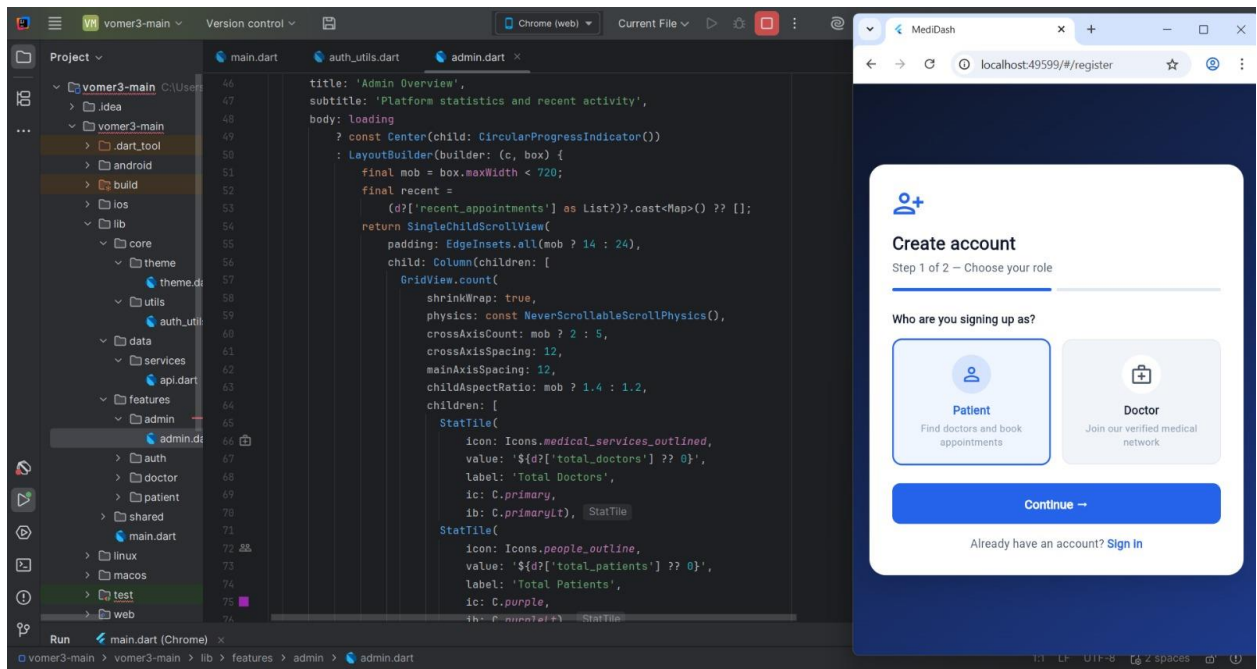


Figure 26: Flutter Code - Admin Code Detail Create Account (Part 1)

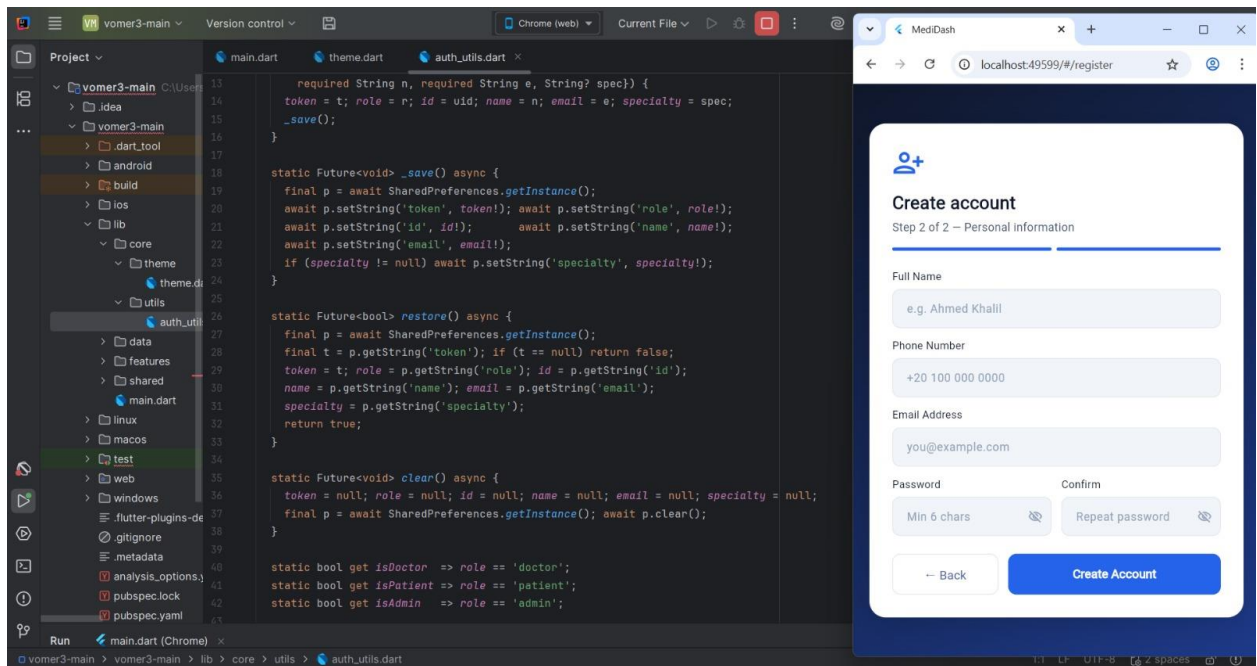


Figure 27: Flutter Code - Create Account (Part 2)

## 5.7 Photos from Website and Mobile application launching

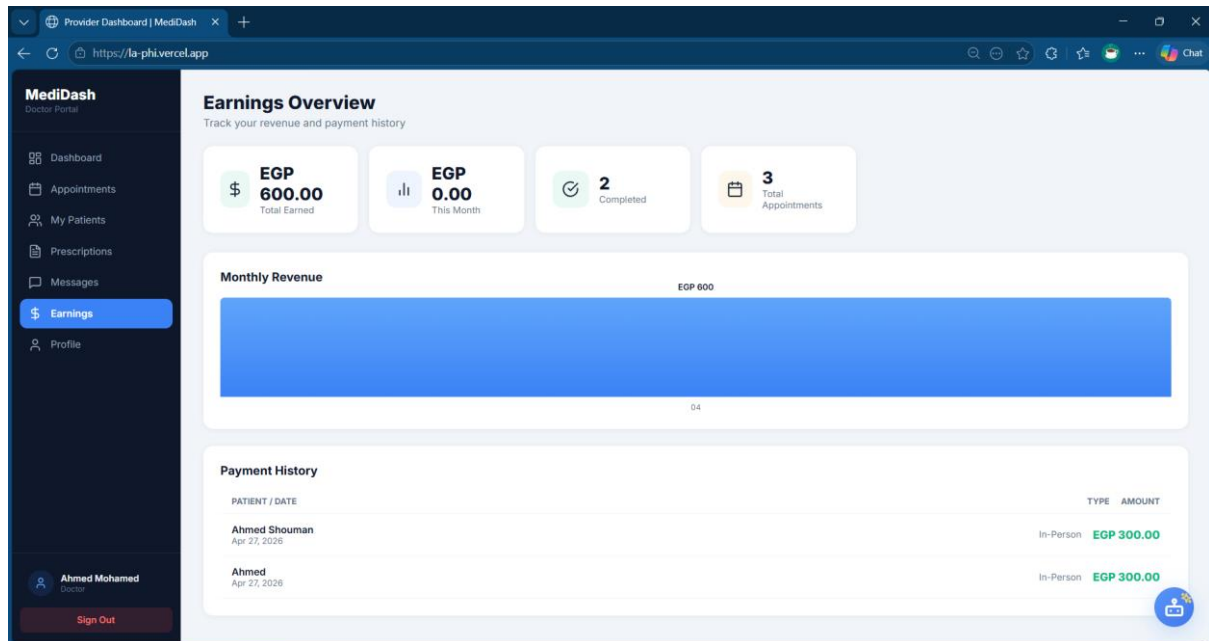


Figure 28: Website – Earning Overview

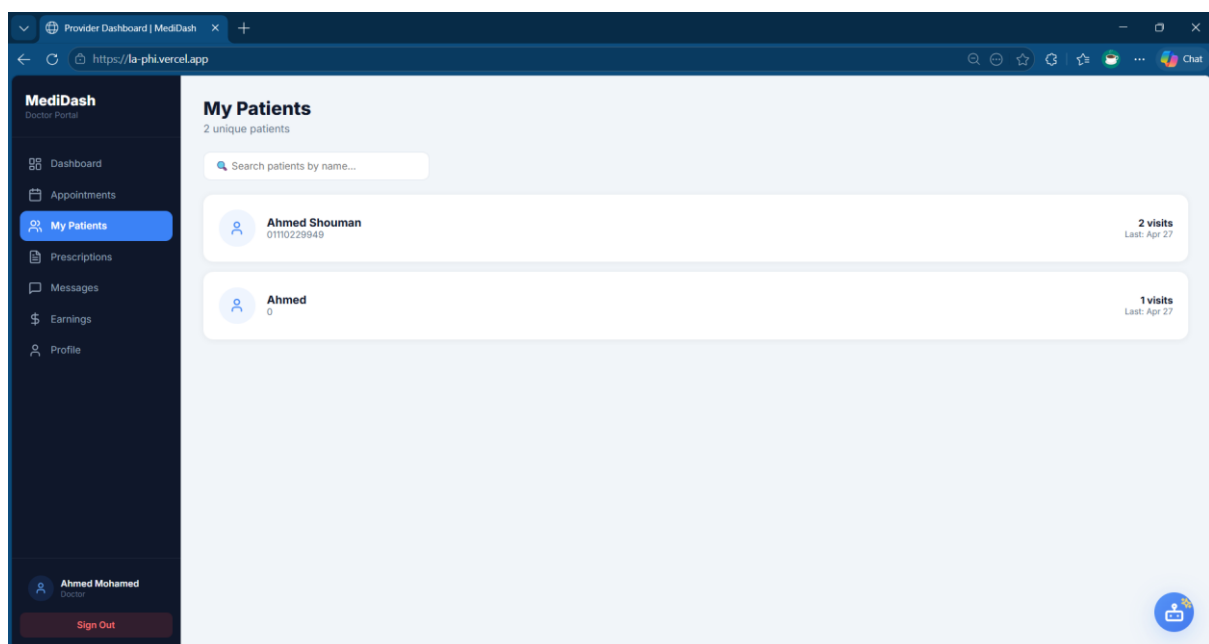
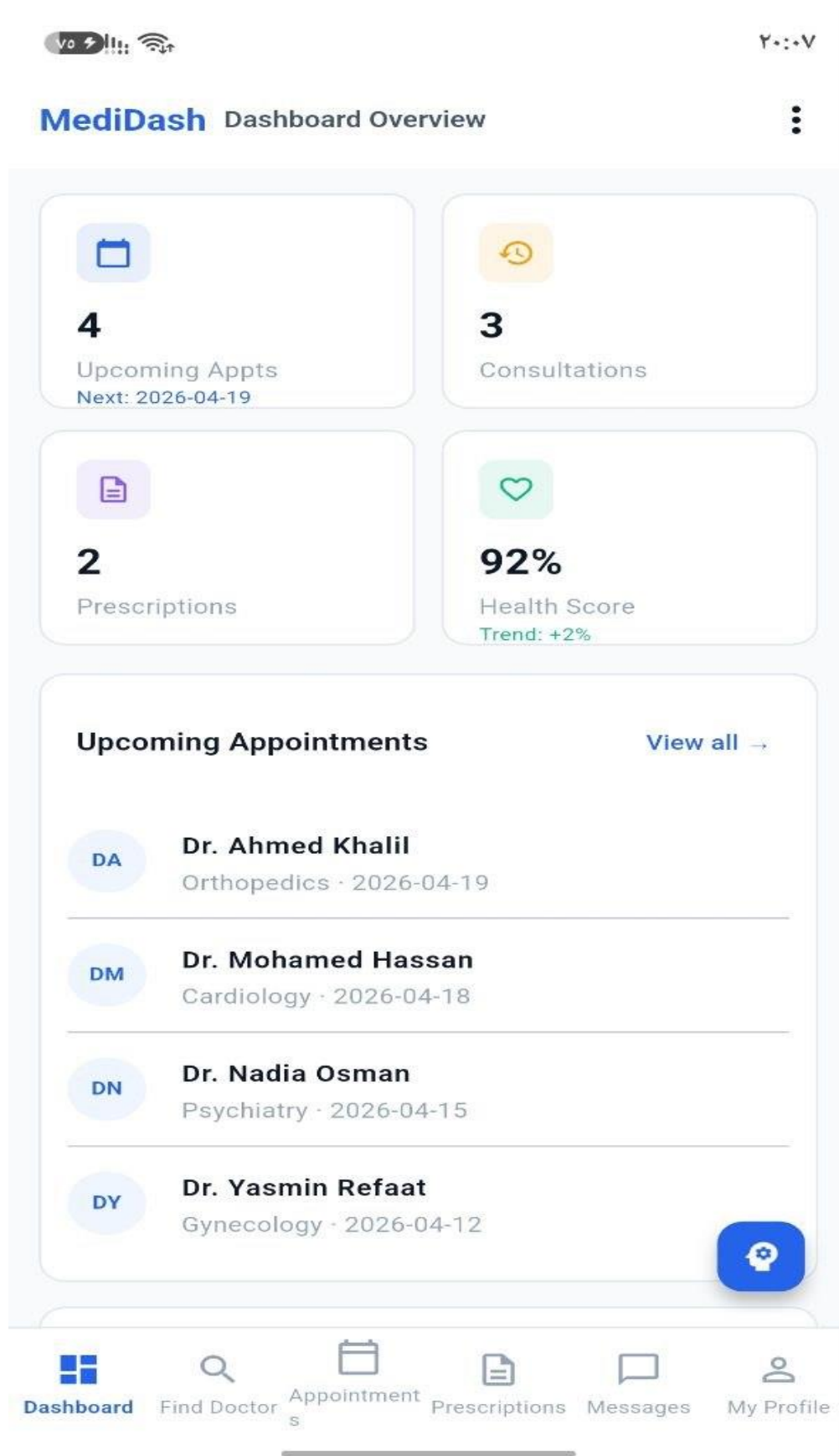
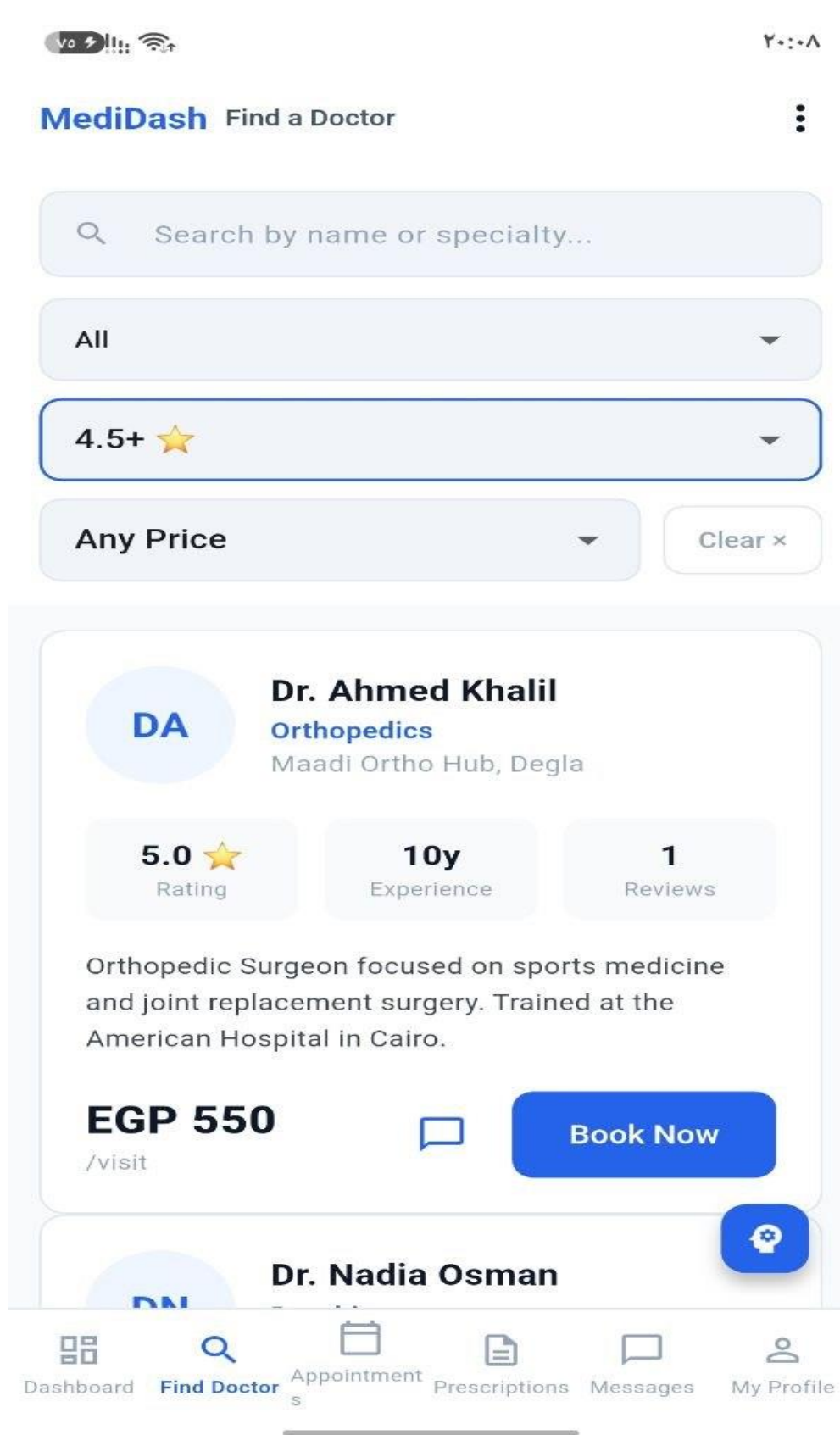


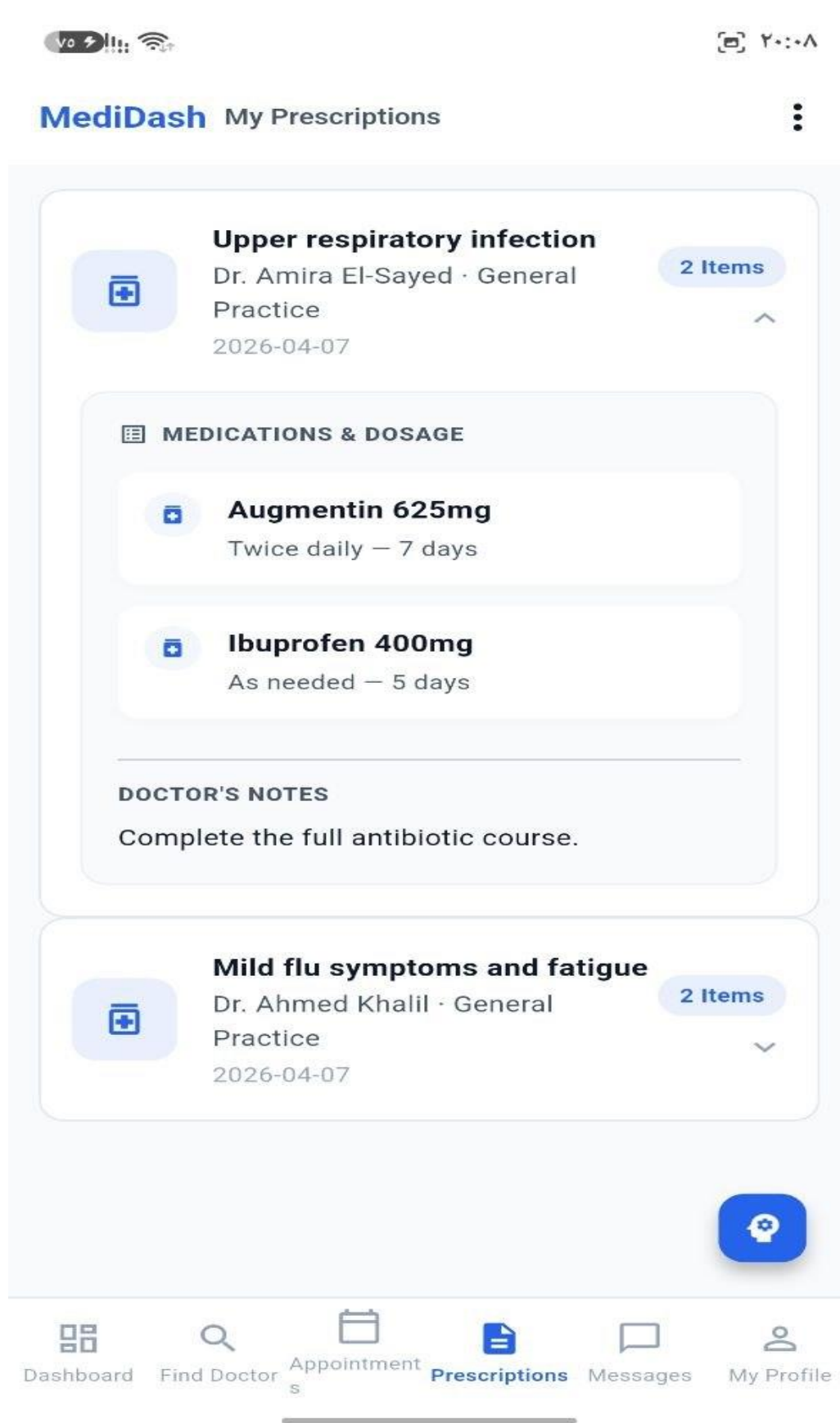
Figure 29: Website – Patient of Doctor



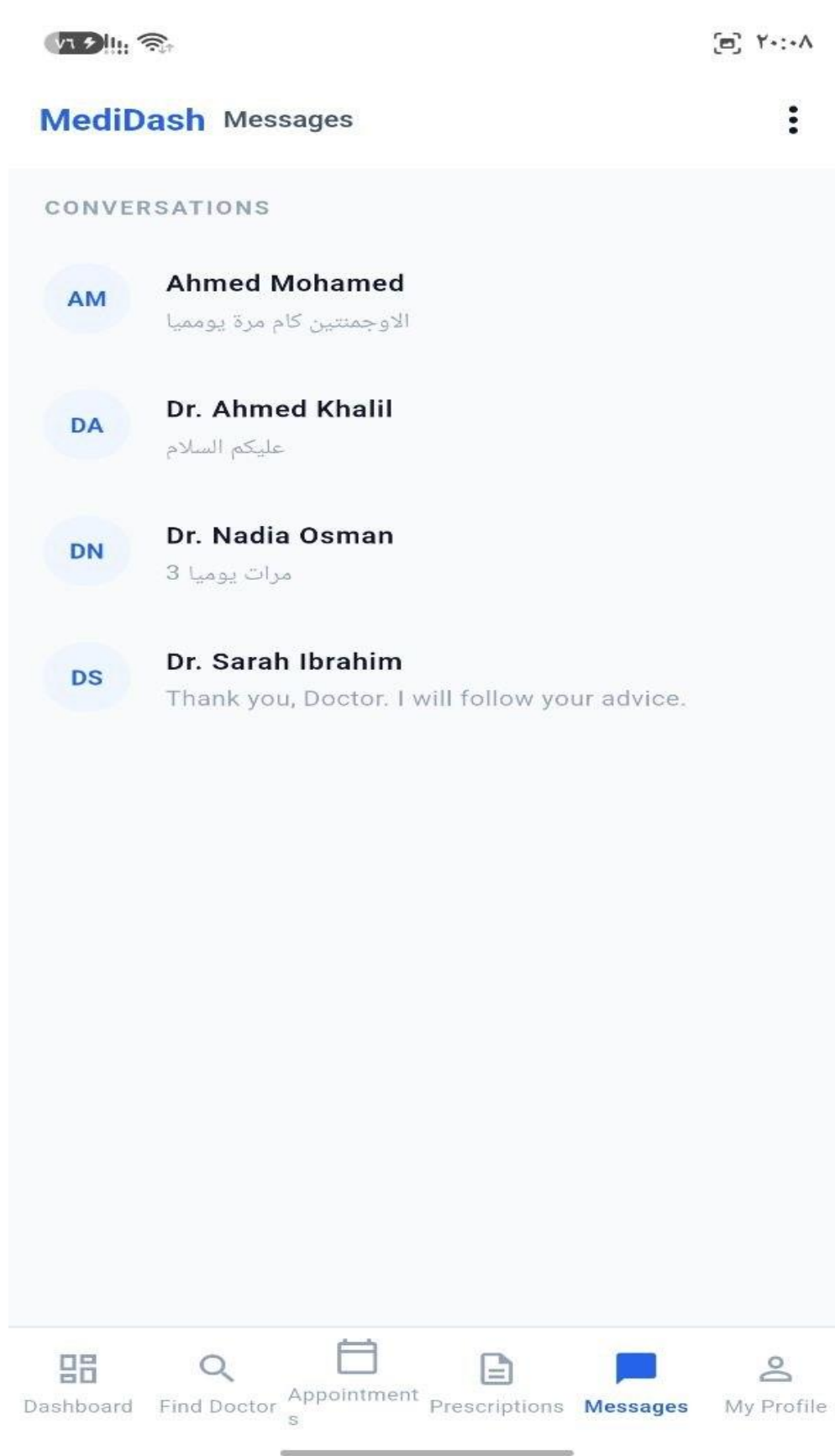
*Figure 30: MobileApp– Patient Dashboard*



*Figure 31: MobileApp– Find Doctor*



*Figure 32: MobileApp– Patient's Prescription*



*Figure 33: MobileApp– Patient message doctor*

MediDash

My Profile

Full Name

Ahmed Shouma

Phone Number

01110229949

Date of Birth

2004-05-25

Blood Type

A+ ▼

Height (cm)

168

Weight (kg)

110

Medical Information

Allergies

no

Chronic Conditions

not found

Emergency Contact

Contact Name

Shouman

Contact Phone

01110229949

Dashboard

Find Doctor

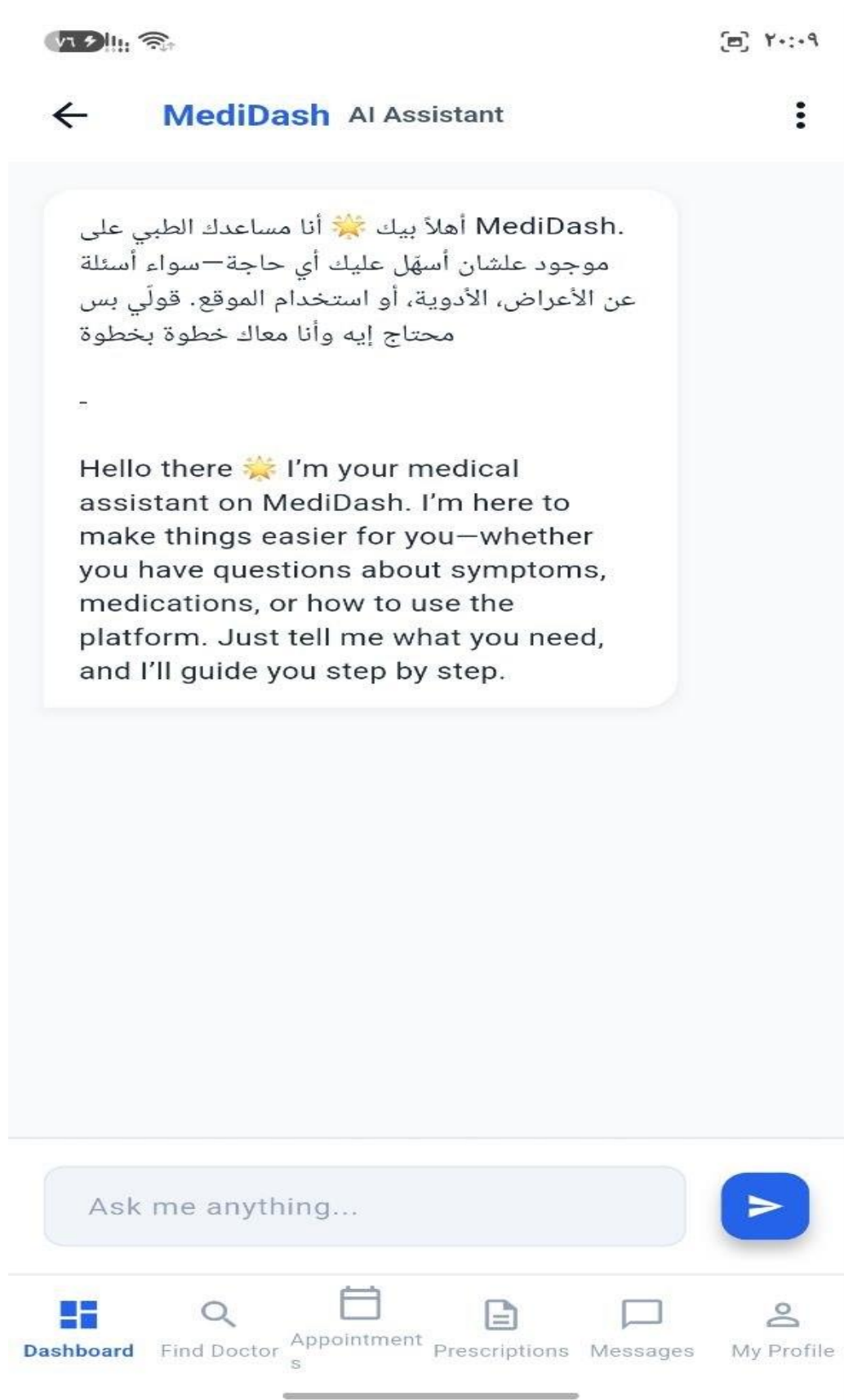
Appointments

Prescriptions

Messages

My Profile

*Figure 34: MobileApp– Patient profile*



**Figure 35: MobileApp– Chat-Bot**



**Figure 36: MobileApp– Patient ask Ai assistant**

---

## Chapter Six : Testing

---

## Chapter Six: Testing

### 6.1 Introduction

This chapter presents the testing process of the Online Medical System to ensure that all system functions work correctly and efficiently. The testing process was conducted to verify system performance, security, reliability, and usability for patients, doctors, and administrators.

### 6.2 Project Description

The Online Medical System is a web-based platform designed to help patients and doctors manage medical services online. The system provides several features, including:

- Patient and doctor registration
- Appointment booking and management
- Medical records management
- Prescription handling
- Notifications and reminders
- Secure data storage

### 6.3 Testing Scope

The testing scope covered all major modules and functionalities:

- User registration and login
- Appointment scheduling and cancellation
- Patient record management
- User roles and permissions
- Database operations
- User interface functionality
- Data security and privacy

## 6.4 Testing Methods

### 6.4.1 Integration Testing

Integration testing was performed to verify the interaction between different modules of the system, such as communication between the user interface and database, integration between the appointment system and notification service, and interaction between doctor dashboards and patient records. All integrated components worked together successfully.

### 6.4.2 System Testing

System testing was conducted on the complete application to ensure all system functions operate correctly, the system performs efficiently, response time is acceptable, and the application remains stable during usage. No critical issues were found during testing.

### 6.4.3 User Acceptance Testing

User Acceptance Testing (UAT) was performed by real users, including doctors and patients. Users reported satisfaction with ease of navigation, fast appointment booking, clear interface design, and accessibility of medical information.

## 6.5 Tools and Frameworks

- Postman for API testing
- MySQL for database management
- Visual Studio Code for development and debugging
- Google Chrome for interface testing

## 6.6 Success Criteria

The system was considered successful when the following conditions were achieved:

- All functionalities worked without errors
- Login and appointment booking operated correctly
- Data was stored accurately in the database
- System response time was fast
- Patient data remained secure
- The system was easy to use

## 6.7 Conclusion

The testing results demonstrated that the Online Medical System operates effectively and satisfies the main system requirements. The system achieved good performance, stability, usability, and security.

## 6.8 Performance Testing

Performance testing was conducted to evaluate the system's ability to handle multiple users simultaneously. Tests included measuring page loading speed, evaluating appointment booking response time, testing system behavior under concurrent users, and monitoring database and server performance. The system could handle normal user traffic efficiently without significant delays or failures.

---

## Chapter Seven : Conclusion

---

## Chapter Seven: Conclusion

### 7.1 Summary of Achieved Results

This project presented the development and implementation of an Online Medical System designed to improve healthcare services through digital technology. The system successfully achieved its main objectives by providing an efficient and user-friendly platform for both patients and doctors.

The developed system allows users to:

- Register and log into the system securely
- Book and manage medical appointments online
- Store and manage patient medical records
- Improve communication between doctors and patients
- Reduce paperwork and save time
- Access medical information easily and quickly

The testing process showed that the system operates correctly and provides good performance, reliability, and security. Overall, the project demonstrated how modern web technologies can enhance healthcare management and simplify medical services.

## 7.2 Possible Extensions and Further Development

Although the current version of the system provides essential medical services, several improvements and future extensions can be added:

- Enhance a video consultation and online meeting features
- Integrating online payment systems
- Further development of the mobile app
- Enhancement and improving AI capabilities
- Supporting multiple languages
- Implementing automatic SMS and email notifications
- Enhancing system security with advanced authentication methods
- Connecting the system with pharmacies and laboratories

These future enhancements can improve user experience, increase system efficiency, and provide more advanced healthcare services to a wider audience.